Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Ibrahim Onuralp Yigit[a,*], Feza Buzluca[a]

[a]*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Assessing distributed system dependability before deployment is hindered by absent runtime telemetry and static code analysis's blindness to asynchronous communication topology. We present Software-as-a-Graph (SaG), a static system analysis framework that constructs typed multigraphs from Architecture-as-Code manifests across five entity types. SaG pairs two parameter-independent pathways: a relation-specific Heterogeneous Graph Transformer (HGT) with Quality-of-Service (QoS) edge encodings forecasting cascade blast radii, and an interpretable ISO/IEC 25010 layer attributing fragility to Availability or Fault Tolerance to guide repairs.

Across twelve synthetic scenarios and five open-source reference systems, all labelled by simulation rather than observed failures: (1) relation typing gains $\Delta\rho = +0.114$ over homogeneous learning under inductive distribution shift, winning 11 of 12 folds ($p = 0.0122$), while in-distribution the two are indistinguishable (-0.023) — typing is an inductive bias for unseen architectures, not added capacity; (2) the QoS edge encoding contributes independently ($+0.054$, 11/12, $p = 0.0093$); (3) against training-free QoS-weighted centrality the model is *not* superior — its nominal $+0.127$ ($p = 0.077$) rests on one fold where that baseline fails outright, falling to $+0.078$ without it; (4) zero-shot transfer to open-source systems reaches $\rho = 0.682$ against ≈ 0.51, but across components that actually propagate failures the mean falls to $+0.180$, negative on two of five systems; (5) the neural forward pass is the cheapest pipeline stage by three orders of magnitude, leaving deterministic graph analysis as the cost that matters. Typed, QoS-aware graph learning thus improves reproducibly on untyped learning, and remains an open question against closed-form centrality.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Distributed systems dependability, Publish–subscribe architecture, Quality-of-Service policies, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. Across diverse domains—from autonomous driving (ROS 2 [1]) and enterprise event streams (Apache Kafka [2]) to cyber-physical backbones (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters—pub-sub decouples producers and consumers in space, time, and synchronization [7]. Components interact indirectly through intermediate message topics and brokers without maintaining direct static references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—such as reliability guarantees, durability, message priorities, and delivery deadlines—to govern how traffic behaves under peak load and network stress.

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

While this architectural decoupling confers elastic scalability and operational flexibility, it creates a formidable **visibility barrier** for system performance, reliability, and computational sustainability:

- **Indirect Failure and Degradation Pathways:** In traditional synchronous architectures (e.g., RESTful HTTP or gRPC), component interactions follow explicit caller–callee invocation paths. In asynchronous pub-sub and event meshes, publishers and subscribers share no direct references. Cascading failures, queue head-of-line blocking, and backpressure propagate across hidden logical paths spanning brokers, shared topics, colocated execution nodes, and shared libraries.
- **Distinct Degradation Mechanisms:** Disturbances in complex distributed systems do not propagate in a uniform manner. They manifest either as *sequential cascades* (e.g., a slow subscriber causing broker queue saturation and upstream backpressure) or as *simultaneous blast radii* (e.g., a shared runtime library crash, memory exhaustion, or host machine outage instantly disabling multiple colocated services). Conventional architectural diagrams and static call graphs fail to represent these multi-layer dependencies.

Addressing these architectural vulnerabilities is most effective and cost-efficient **prior to deployment**, during design and Continuous Integration / Continuous Delivery (CI/CD), adhering to established foundational principles of dependable computing [8]. However, at design and build time, **no runtime telemetry, distributed tracing, or operational logs exist**. Consequently, software architects, performance engineers, and Site Reliability Engineers (SREs) face two fundamental questions without operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural repair (such as replicating a message broker, decoupling an over-subscribed topic, or sandboxing a shared library) will most effectively eliminate that risk?*

The same questions bear on computational sustainability, though we are careful about the form of the claim. Analysing an architecture from a manifest requires no provisioned cluster, no running services and no fault-injection harness — the resource it eliminates is infrastructure rather than CPU time. What it does not do is compute less: the deterministic structural analysis this framework depends on is expensive, and on our own corpus it costs substantially more than the discrete-event simulation it produces labels from (Sections 7.5 and 8.2). We measure wall-clock latency rather than energy counters, and we make no claim that the pipeline is computationally cheap in absolute terms. The narrow and supported statement is that the *learned* component is negligible within it — a 56 ms forward pass against minutes of feature extraction — so applying graph neural networks to pre-deployment analysis does not itself introduce a meaningful cost.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** We forecast dynamic cascading failure blast radii and identify critical components using a data-driven, relation-specific model over learned topological representations. Closed-form topological metrics capture broad connectivity cheaply; whether they also resolve multi-hop, relation-dependent cascade spread across heterogeneous channels is an empirical question, which we test directly against such a baseline (Section 7.1). Our predictive pathway is trained and evaluated against independent simulation ground truth as a ranking and critical-set identification model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist indicates *where* risk lies, but not *how to fix it*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [9] and ISO/IEC 25019 [10]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing, for instance, an unreplicated single point of failure from a high-coupling maintainability bottleneck—to guide concrete repairs. It serves strictly as an attribution model, not a ranking model.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other's output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this independence allows SaG to identify components that are structurally central yet operationally low-impact—a nuanced diagnosis unattainable by either pathway alone.

Existing software engineering approaches fail to bridge what we define as the **Architecture–Code Gap**: *a distributed system can have pristine, bug-free source code within each individual service, yet remain fragile to catastrophic global outages caused by hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts.* Although classical architecture evaluation methods such as the Architecture Tradeoff Analysis Method (ATAM) [11, 12] identify architectural risks, and software studies analyze architectural technical debt [13] and bad smells [14], they rely on manual stakeholder elicitation rather than quantitative structural learning. Prevailing automated paradigms each leave part of this gap unaddressed. Static code analysis [15, 16, 17, 18] inspects complexity, cohesion and coupling *within* a service but cannot observe message queues or cross-host propagation. Runtime chaos engineering [19] injects real faults but needs a provisioned cluster, carries operational risk, and arrives after the architecture is fixed. Homogeneous centrality metrics [20, 21, 22, 23] flatten the system into an untyped graph in which a message topic, a shared library and an execution host are indistinguishable. Section 2 develops each of these in turn.

Furthermore, while machine learning has demonstrated remarkable success across software engineering, contemporary AI approaches applied to system dependability often function as uninterpretable black boxes. Deep neural models frequently output scalar risk scores or latent embeddings without providing transparent, actionable rationales for their predictions. In mission-critical software engineering, an opaque risk score is inadequate: developers and SREs cannot refactor code or reconfigure infrastructure without understanding *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming the black-box AI challenge, this work introduces **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and outputs per-relationship criticality alongside multi-task quality outputs (Section 4).
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), pointing to distinct repairs. Because it is a linear, propagation-free aggregate by design, it explains *why* a component is vulnerable rather than how far a cascade travels; its standalone rank correlation is correspondingly modest (Section 7.1).

To ensure methodological rigor, SaG enforces a strict **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical graph G_{analysis}, while ground-truth failure impacts are generated by independent discrete-event simulators operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4).

Figure 1 shows how the two pathways relate. The predictive pathway is the primary one and the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not — and that oracle is strictly an offline training-and-validation component, never a dependency of online inference. The explanation layer then characterises what the predictor flagged and the remediation that implies. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor ranked. The remediation guidance closes a loop of its own, in which each candidate edit is re-simulated on its own mutated copy of $G_{\text{structural}}$ and kept only if it beats the simulator's seed-to-seed noise.

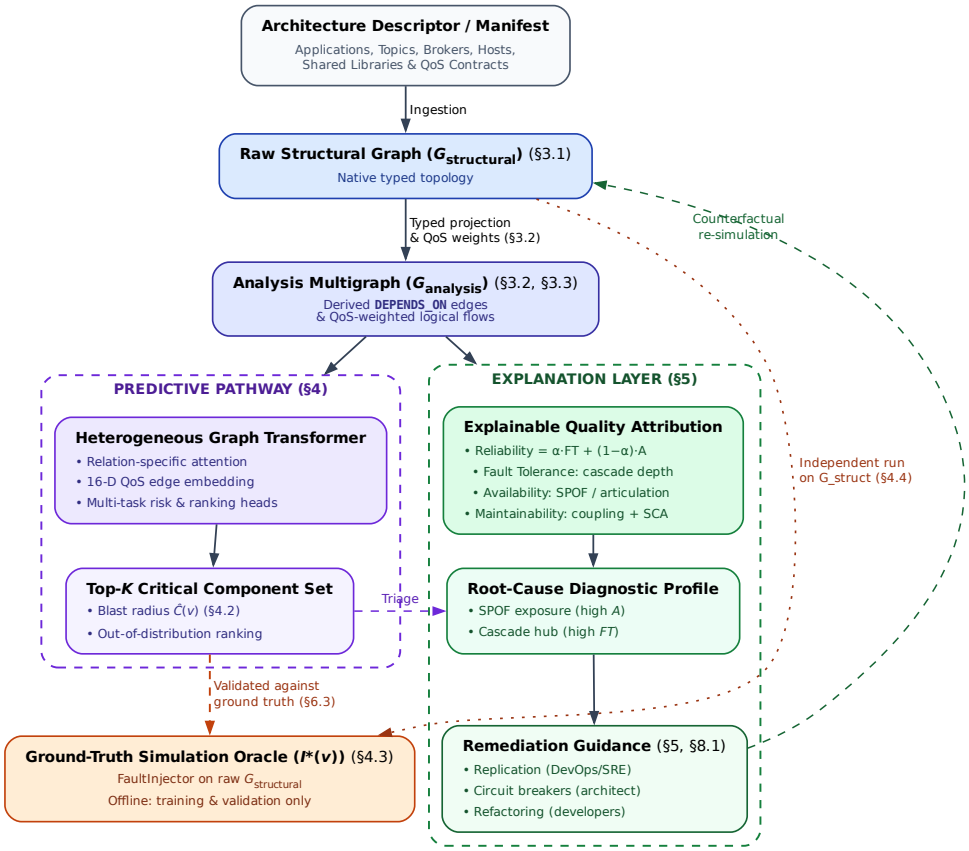

Figure 1: End-to-end architecture of the SaG framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted `DEPENDS_ON` projection → typed node features) feeds two pathways that share no parameters: the predictive pathway (Section 4), which emits a ranked critical set and per-relationship criticality, and the explanation layer (Section 5), which emits a standards-grounded quality profile. The simulation oracle scores only the former and runs on $G_{\text{structural}}$ alone.

Rationale for Graph Learning vs. Direct Simulation. Since discrete-event simulation $I^*(v)$ defines ground-truth criticality here, it is fair to ask why train a graph model at all rather than run simulation sweeps or closed-form heuristics. Three reasons motivate the design, and Section 7.1 tests them adversarially. A trained model scores entity types no simulator sweep was run for, since message passing generalizes across labelled and unlabelled entities alike. Cascade simulation is stochastic and seed-sensitive (label standard deviation reaches 0.416), whereas a trained model learns a smooth, threshold-marginalized surrogate that re-scores an already-analysed architecture cheaply. And dynamic simulators need runnable containers or

communication harnesses, whereas graph learning scores Architecture-as-Code manifests before any runtime infrastructure exists. A fourth motivation — that neither a simulator nor an unaugmented GNN returns a root cause in standardized quality terms — motivates the explanation layer rather than the predictor, and is taken up in Section 5. Whether these motivations are borne out empirically is a separate question, and Section 7.1 answers it only partly in the framework's favour.

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding and Robustness): *Do middleware Quality-of-Service contracts carry signal a purely structural score discards, do the framework's simulation oracles agree with one another, and are the reported orderings robust to the free parameters of the scorer and of the ground truth?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2), cloud-native microservices, smart home IoT, and industrial edge computing?*

RQ5 (Analysis Cost): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that footprint, and how does it compare against the discrete-event simulation it is intended to displace?*

1.5. Key Contributions

This paper presents four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code alone, with a 16-dimensional edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Our central empirical claim concerns cross-architecture transfer: with matched training sets, substrate, depth and model selection, typed learning leads untyped learning by $\Delta\rho = +0.114$ (0.695 vs. 0.581) under inductive distribution shift, winning 11 of 12 folds ($p = 0.0122$; Section 7.2), and the unweighted pair replicates it at $+0.147$ (11/12, $p = 0.0010$). Crucially, this is an inductive bias rather than a capacity advantage: *in*-distribution, typed and untyped learning are indistinguishable and the best mean belongs to the untyped model (-0.023, $p = 0.813$; Section 7.2). Relational typing earns its place precisely where the architecture is unseen — which is the only regime a pre-deployment gate ever operates in. The 16-D QoS edge encoding contributes independently, improving out-of-distribution ranking by $+0.054$ within the typed architecture (11/12, $p = 0.0093$) and $+0.088$ within the untyped one (Section 7.3.1). Simultaneously, we establish the honest empirical boundary: against an unparameterized QoS-weighted centrality baseline (`Topo-QoS`), out-of-distribution ranking is *not* surpassed — $\Delta\rho = +0.127$ (9/12, $p = 0.077$), a margin carried almost entirely by one fold on which that baseline fails outright, and which falls to $+0.078$ ($p = 0.148$) when the fold is excluded (Section 7.1).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures, supplying the typed substrate the predictor consumes (Section 3).

3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns the predictor's ranked output into an actionable diagnosis, separating single-point-of-failure exposure from error-propagation depth—two distinct failure modes requiring different repairs (Section 5).
4. **Empirical Benchmark, Real-World Evaluation, and Cost Characterization:** An evaluation across eleven synthetic topologies (2,387 components) and five open-source reference systems (351 components) under strict graph-view separation, establishing both where typed graph learning helps and the boundary where it does not, with a per-stage cost profile that locates the pipeline's cost in its deterministic graph-analysis stage rather than in the learned model, and that measures the framework's own gate against its simulation oracle — finding the gate the more expensive of the two, and withdrawing on that basis the computational-efficiency claim made in the conference version (Sections 6–7).

Relationship to the authors' prior work. An earlier, shorter version was presented at a peer-reviewed conference [24], covering the preliminary typed multigraph formulation and the deterministic quality-attribution model on the synthetic corpus alone. This manuscript is a substantially extended version meeting the JSS extension policy. New here are: the entire predictive pathway (the HGT, its 16-D QoS edge encoding, the multi-task masked-loss heads, and every learned result in Section 7); the inductive LOSO protocol and cross-architecture transfer analysis (Section 7.2); zero-shot evaluation on five open-source reference systems (Section 7.4); the cost characterization answering RQ5 (Section 7.5); the four-oracle convergent-validity taxonomy and graph-view separation (Sections 4.3–4.4); global sensitivity analysis over all ten weight constants (Section 7.3); and the homogeneous-versus-heterogeneous comparison under substrate parity (Section 7.2). Material retained from the conference version is limited to preliminary formalisms in Sections 3 and 5, both restructured and expanded. No companion manuscript from this work is under consideration elsewhere.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work on distributed systems dependability, performance engineering, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model, the dependency projection rules, and the typed node features consumed by both pathways. Section 4 presents the Heterogeneous Graph Transformer, its multi-task heads, the simulation oracles that supply its labels, and the input–label independence guarantee. Section 5 introduces the interpretable RM explanation layer. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ5. Section 8 discusses architectural implications, performance and computational sustainability, threats to validity, limitations, and concluding remarks.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [19] inject faults or latency into staging or production clusters to observe degradation and recovery. While runtime fault injection delivers operational validation that no static method can match, it requires a fully provisioned cluster, carries the risk of real service disruption, and incurs severe computational and carbon costs from hours of cluster execution per sweep — precluding its use during architectural design or lightweight commit-level CI/CD.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities and performance degradation directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned. From a green software engineering perspective, this enables zero-runtime, computationally sustainable architectural gating within CI/CD pipelines without the energy footprint of running clusters. Furthermore, by identifying cascading failure hubs and single points of failure at design time, pre-deployment analysis prevents runtime restart storms, connection-pool thrashing, and tail-latency explosions that waste massive compute cycles and data center power in production.

Architecture-Based Reliability Prediction. Predicting dependability from an architectural description before deployment is not a new ambition, and SaG should be read against the tradition that pursued it analytically. Cheung's absorbing-Markov-chain model [25] derives system reliability from component reliabilities and a transfer-of-control graph; Goseva-Popstojanova and Trivedi [26] systematize the state-based, path-based and additive families that followed, and Immonen and Niemelä [27] survey the resulting methods from the architectural perspective. Model-driven descendants such as the Palladio Component Model [28] and layered queueing networks [29] predict performance and reliability from parameterized component models with well-understood solution techniques.

These methods are complementary to ours rather than superseded by it, and the distinction is one of required inputs rather than of accuracy. They need per-component failure probabilities, transition probabilities or service demands — parameters that are themselves estimated from operational profiles, measurement or expert judgement, and that are unavailable in the setting we target (a manifest at commit time, with no telemetry). SaG asks a narrower question in exchange: not what the system's reliability *is*, but which components' failures would propagate furthest through the declared topology. Where those parameters can be obtained, an analytical model answers a stronger question than a ranking does, and we make no claim to displace it.

Data-Driven Failure Prediction and Root-Cause Analysis in Microservices. A large recent literature localizes faults in microservice systems from operational data. Seer [30] and Sage [31] predict and debug QoS violations from traces and hardware telemetry; MicroRCA [32] and TraceRCA [33] localize root causes over service-dependency and trace graphs; DeepTraLog [34] and Eadro [35] combine traces, logs and metrics under graph-based deep models. This line is the closest methodological neighbour to our predictive pathway, and it consistently outperforms what a purely static analysis can achieve — because it observes the running system. That is precisely the boundary: every one of these approaches requires a deployed system emitting traces, logs or metrics, and therefore cannot answer a question posed at design or pull-request time. SaG occupies the pre-deployment complement, and accepts a correspondingly weaker evidential basis: simulated rather than observed failures, and topology rather than behaviour.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [15]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [16], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [17, 18], and code duplication to flag internal code smells and defect-prone modules [36, 37, 38, 39]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

To bridge this "Architecture–Code Gap," **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message

topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [40, 41] and architectural technical debt [42] early during continuous integration (CI/CD) [43, 44], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [9] and the **ISO/IEC 25019:2023** Quality-in-Use model [10]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Faultlessness, Availability, Fault Tolerance and Recoverability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Reusability, Analysability, Modifiability and Testability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

SaG operationalizes a strict subset of these: Availability and Fault Tolerance under Reliability, and Modularity, Modifiability and Analysability under Maintainability (Section 5.1). Faultlessness, Recoverability, Reusability and Testability are not derivable from deployment topology alone and are outside the scope of this work.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [45, 46]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [47] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure mathematical soundness in weighting models. This study applies AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, in conjunction with learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [20, 22], articulation points, and PageRank [21, 23]. Foundational studies on network robustness [48], cascading overloads [49], and interdependent networks [50] model how disruptions propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality scalar cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating cascade hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared library, and a physical execution host.

To overcome the limits of hand-engineered metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [51], DrBC [52], and PowerGraph [53]). However, most available models rely on **homogeneous message passing** (GCN [54], GraphSAGE [55], GAT [56]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries and fail to generalize out-of-distribution.

Heterogeneous Graph Neural Networks (RGCN [57], HAN [58], HGT [59], MAGNN [60]) resolve this by employing relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [59] to preserve typed relational semantics when forecasting cascading failure blast radii and performance degradation.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [61] and PGExplainer [62], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \tag{1}$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \tag{2}$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \to \mathcal{T}_V$ and $\tau_E : E \to \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \to [0, 1]$ and $w_E : E \to [0, 1]$ are weighting functions representing entity criticality and connection strength.

Table 1 summarizes the five entity types and six structural edge types formalized in the SaG model, along with their semantics and representative concrete distributed systems implementations.

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	**Architectural Role**	**Concrete System Examples**
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	`/sensor/lidar`, `orders.payment.completed`
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	`librdkafka`, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	**Direction**	**Semantic Meaning**
`PUBLISHES_TO`	App/Library → Topic	Component publishes messages to topic
`SUBSCRIBES_TO`	App/Library → Topic	Component consumes messages from topic
`ROUTES`	Broker → Topic	Broker manages and routes topic traffic
`RUNS_ON`	App/Broker → Node	Process is hosted on physical/virtual host
`CONNECTS_TO`	Node → Node	Physical network link between hosts
`USES`	App → Library	Application links to shared library dependency

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a `RELIABLE` topic with `TRANSIENT_LOCAL` durability binds communicating services substantially more tightly than a `BEST_EFFORT` telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75,\, 0.15,\, 0.10) \tag{3}$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24,\, 0.62,\, 0.14) \tag{4}$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and transport priority both govern in-flight delivery quality, with reliability receiving higher weight because unconditional delivery guarantees precede message scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix with a small, non-zero consistency ratio ($CR \approx 0.016 \leq 0.10$). The modulators are logarithmically compressed and clamped to $[0, 1]$: $\text{SizeNorm}(t) = \min(1.0, \log_2(1 + \text{bytes})/20)$ (a 1 MiB design envelope, representing the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \min(1.0, \log_{10}(1 + \text{Hz})/3)$. The final weight $w(t)$ is clamped to $[0.01, 1]$, ensuring that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`) inherits $w_E(e) = w(t)$ along with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination. Sweeping it over the full simplex changes the induced ordering of $w(t)$ by at most $\rho = 0.076$ and downstream rank correlation by at most 0.020, so it is a documented convention rather than a tuned parameter (Supplementary Section S1).

3.2.1. Logical Dependency Projection (`DEPENDS_ON`)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, `DEPENDS_ON`, directed from *dependent* to *dependency* ("if target fails, source is impacted"), according to the six projection rules detailed in Table 2:

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [63, 64, 65]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [66] to combine the consuming Application's and the shared Library's vertex weights, balancing caller and dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent → Dependency)	Derived Weight (w)
1	`app_to_app`	Subscriber → Publisher (via shared Topic, incl. transitive `USES`)	$1 - \prod_{t \in T}(1 - w(t))$
2	`app_to_broker`	Publisher/Subscriber → Broker routing its topics	$1 - \prod_{t \in T}(1 - w(t))$
3	`node_to_node`	Host → Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	`node_to_broker`	Host → Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	`app_to_lib`	Application → Shared Library it `USES`	$H(w_V(\text{app}), w_V(\text{lib}))$
6	`broker_to_broker`	Broker ↔ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{node})$

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It does not imply that one broker functionally depends on another, but rather that two brokers colocated on the same host share that host's physical failure domain. This follows the same simultaneous-blast principle as Rule 5, which is why the derived weight equals the shared Node's weight and the relation is bidirectional. In production middleware deployments, colocated brokers compete for host resources (CPU cores, page cache, file descriptors, and NIC bandwidth); a host outage takes down all colocated instances simultaneously. Operational best practices for Kafka, RabbitMQ, and EMQX recommend distributing brokers across fault domains. Rule 6 does not model directional intra-cluster broker coupling (e.g., partition replication, controller quorum election, federation, or shovel links), which do not require physical colocation; extending the schema to capture these interactions is reserved for future work. Rule 6 applies in four of the eight scenarios forming the detection benchmark subset — the seven core synthetic topologies plus the ATM case study, the subset identified in Table 6 — and contributes only 12 directed edges across them. Because the simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis}.

Figure 2 illustrates this duality on a running example, contrasting the raw structural graph against the derived `DEPENDS_ON` projection.

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [67].

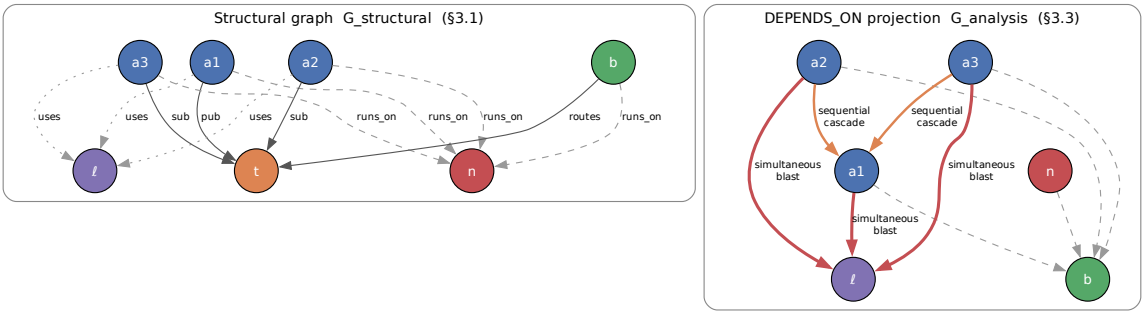

Figure 2: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis}: the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load) produced by the deterministic analysis stage whose cost is characterized in Section 7.5. Type-specific features extend that block:

- **Application (23 dims):** indices 18–22 add source-code metrics from SCA — lines of code, cyclomatic complexity, Martin's instability $I_{\text{code}} = C_e/(C_a + C_e)$ [68], Lack of Cohesion in Methods, and the composite Code Quality Penalty (CQP).
- **Library (25 dims):** the Application block plus two library-specific blast-radius drivers (23–24): the normalized size of the transitive reverse-`USES` closure, and the normalized count of distinct subscribers reachable from topics published within that closure.
- **Broker (19 dims):** index 18 is normalized queue buffer capacity.
- **Topic (22 dims):** indices 18–21 are publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node (20 dims):** indices 18–19 are normalized CPU core allocation and physical memory.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on neighbor count, but through architectural relations and dependencies extending multiple hops beyond the initial fault. Whether a closed-form combination of standard centrality metrics can capture these compound dynamics is an empirical question rather than a settled one. The primary predictive pathway of Section 1.2 therefore employs a learned graph model, and Section 7.1 evaluates it against exactly such a closed-form baseline — which, on out-of-distribution ranking, it does not significantly surpass.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input–label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [59], implemented within PyTorch Geometric [69], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e in G_{analysis}; indices 2–8 one-hot encode the seven structural and derived relations (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`, `DEPENDS_ON`). Indices 9–15 encode middleware QoS parameters on `PUBLISHES_TO` and `SUBSCRIBES_TO` edges (zeroed elsewhere). Four of these dimensions are active in our evaluation corpus: reliability (0 best-effort, 1 reliable); durability (0 volatile, 0.5 transient-local, 0.6 transient, 1 persistent); message priority (0, 0.33, 0.66, 1); and a heterogeneity flag set when the edge's QoS triple departs from its scenario's modal profile. The remaining three indices (deadline-active flag, $\log_{10}(1 + \text{deadline\_ns}/10^6)$, and $\log_{10}(1 + \text{max\_blocking\_ms})$) are architectural schema provisions designed to accommodate hard real-time DDS and ROS 2 microsecond profiles, and are zero-initialized in the current benchmarks.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}} e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D-dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}\big(\text{GELU}(W_{\tau(v)} x_v)\big) \tag{5}$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention across H heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u) W_{\text{att},\phi(e)} Q(\tilde{h}_v)^\top}{\sqrt{D/H}} \right) \tag{6}$$

$$\text{Msg}(u, e, v) = V(u) W_{\text{msg},\phi(e)} \tag{7}$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream backpressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G^\top_{\text{analysis}}$).
4. **Residual Aggregation and Layer Normalization:** Target node representations are updated across layers $l \in \{1, \ldots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm}\left(h_v^{(l-1)} + \text{Dropout}\left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \tag{8}$$

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4}, and dropout probability $p = 0.10$ applied post-attention. Learning rates follow a cosine annealing schedule with warm restarts (CosineAnnealingWarmRestarts, $T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\min} = 3 \times 10^{-6}$). Training executes for a maximum of 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss over labeled nodes. Inductive subgraphs are processed per scenario using full-graph inductive packing without mini-batch subsampling, with validation masks isolating held-out nodes to prevent information leakage across data splits. Five independent random seeds $\{42, 123, 456, 789, 2024\}$ are evaluated across all runs, redrawing both partition masks and initializations. *Selection protocol:* the architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate and schedule) follow values conventional for HGT [59]. The loss coefficients of Equation (9) have no such precedent — the objective is bespoke to this task — and were set by judgement and left untuned; we state this rather than appeal to a convention that does not exist for a five-term multi-task loss. Neither group was tuned against the in-distribution test split or the LOSO folds; no search over them was performed there. The real-world evaluation of Section 7.4.1 is a separate case and is documented separately: it runs at a different depth and epoch budget from every other learned result in this paper, and Section 7.4.1 states that configuration and how it was arrived at. This avoids selection leakage, at the cost of leaving open whether either family is reported near its own optimum — a comparison between untuned configurations, which we state rather than treat as a like-for-like optimum comparison.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \tag{9}$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE ranking loss [70], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}\big([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}}\big)$ regresses predicted heads toward the diagnostic pathway's baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via `FaultInjector`) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \tag{10}$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

What the reliability label is. The surviving term deserves to be stated plainly, because it is weaker than a multi-task objective normally implies. `FaultInjector` emits a single scalar per component, and the label extractor assigns that same scalar to both the composite and the reliability columns: $R^*(v) = I^*(v)$ identically. $\mathcal{L}_{\text{dimension}}$ under $m = [1, 0]$ therefore regresses $\hat{R}$ toward the very target $\mathcal{L}_{\text{composite}}$ regresses $\hat{I}^*$ toward. The two terms are not redundant — they train separate heads, and $\hat{R}$ re-enters the composite head as an input ($\hat{I}^* = \sigma(\text{MLP}_C(h_v \parallel \hat{R} \parallel \hat{M}))$), so the term acts as an auxiliary path to the same supervision rather than as a second source of information. But no dimension of the label is decomposed by this oracle: a reliability score distinct from overall impact would require an oracle that separates fault-tolerance from availability, which $I^*(v)$ does not. We report the objective as implemented rather than describing it as multi-dimensional supervision it does not receive.

4.2.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \tag{11}$$

where $M_{\text{static}}(v)$ is obtained directly from the structural analyzer's maintainability baseline, combining learned dynamic reliability with static source-code maintainability. Because maintainability is unobserved in dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly, while domain reweighting sensitivity is evaluated against the static RM baseline in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via `FaultInjector`, it crashes component v, propagates outages across dependent topics, brokers and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population, each subscriber contributing the unweighted mean loss of the topics it subscribes to. The denominator is the full set of subscribers in the intact graph: subscribers that themselves fail during the cascade are retained in the average rather than excluded from it. A topic's feed loss is the fraction of its publishers that have failed (for a topic with no publisher, the fraction of its failed routers), scaled by a QoS ladder ($\times 1.2$ `RELIABLE`, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. The implementation admits a per-publisher rate weighting, but publication rates are declared per topic rather than per publisher throughout our corpus, so every publisher of a topic carries the same rate and the rate-weighted form reduces exactly to this publisher fraction. This is the **primary continuous target label** throughout.

- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$)**, via `FailureSimulator`:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \tag{12}$$

  each term weighted by operational severity $s(t) = w(t) \cdot \text{rate}(t)$. These four coefficients are declared, not fitted, and are not among the constants screened in the supplementary sensitivity analysis — a gap we note because I_{comp} supplies the labels for Table 11. rate(t) reads from a runtime telemetry profile when one is attached; none is attached anywhere in this paper, so $s(t) = w(t)$ throughout and the rate channel should be read as an interface for deployment-time calibration rather than an active term. Publication frequency and payload size still reach $s(t)$ through $w(t)$'s own modulators (Section 3.2).

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$)**, via `MessageFlowSimulator` on SimPy [71]: simulates emission rates, stochastic latencies, broker buffer saturation and queue drops under fault injection, extracting the drop in delivered message rate to surviving consumers.

- **Change-Propagation Oracle ($I_M(v)$)**, via `ChangePropagationSimulator`: a deterministic reverse-dependency traversal quantifying maintenance change impact, $I_M(v) = 0.45\,\text{ChangeReach}+0.35\,\text{WeightedChangeImpac}$ $0.20\,\text{NormalizedChangeDepth}$.

- **Relationship (Edge) Removal Oracle ($I_{\mathbf{edge}}(u,v)$):** the systemic impact of severing one dependency while both endpoints stay operational. Writing $\bar{I}_{\text{comp}}(G)$ for the mean composite impact over G:

$$I_{\text{edge}}(u,v) = \bar{I}_{\text{comp}}\big(G \setminus \{(u,v)\}\big) - \bar{I}_{\text{comp}}(G) \tag{13}$$

Topic Criticality Label Masking. `FailureSimulator` can blend a declared `Topic.criticality` into its severity term; this is disabled, because that field is a GNN input feature (`topic_qos_criticality_ord`) and consuming it would score the predictor against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^*, I_{comp}, I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ **(`FaultInjector`) as the primary oracle** for all predictive ranking results (Tables 7–9, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe, and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across seven benchmark scenarios and five random seeds on the Application population, the mean Spearman rank correlation is $\rho = 0.907$ for (I_{dyn}, I^*), $\rho = 0.425$ for (I_{comp}, I^*), and $\rho = 0.427$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The strong rank agreement ($\rho = 0.907$) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms. However, agreement on the top-K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.49 for the strongest pair and 0.24–0.28 for the two I_{comp} pairs, vs. 0.111 expected by chance), highlighting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

4.4. Input–Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.

- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (`FaultInjector`, `FailureSimulator`, `MessageFlowSimulator`).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

What this guarantee does and does not establish. The separation rules out circular feature construction: no predictor can read a transformation of the quantity it is scored against. It does not establish statistical independence between features and labels, and we do not claim it does. G_{analysis} is a deterministic projection of $G_{\text{structural}}$ (Section 3.2), so the labels are — up to simulator seed — a deterministic function of the same topology the features are computed from. Two consequences follow, and both bound the results of Section 7.

First, $I^*(v)$ is itself a topological functional: a breadth-first reachability computation over $G_{\text{structural}}$ scaled by a QoS ladder. The predictive task is therefore to recover a closed-form function of a graph from features of that same graph. Read that way, it is unsurprising that an unparameterized centrality score is competitive with a trained model (Section 7.1); the parity we report there is the expected outcome of the setup rather than a surprising failure of graph learning, and we flag it here so the reader does not have to infer it.

Second, and more restrictively, no result in this paper is validated against an observed failure. Every label — on synthetic topologies and on the five open-source systems alike — is simulator-derived. What the

evaluation can establish is whether a learned model recovers a simulator's ordering on architectures it was not trained on. Whether that ordering corresponds to which components actually fail in production is a question this design cannot answer, and Section 8.3 treats it as the study's principal construct-validity threat.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardized quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck. These diagnoses call for distinct architectural repairs: replicating the host or broker, decoupling the topic, or refactoring the module. This section presents the layer supplying that diagnosis. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [9], **ISO/IEC 25019:2023** (Quality-in-Use) [10], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [72], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system's capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. The two are separated because they imply different repairs, not because they predict different runtime quantities: this layer attributes structural fragility and does not forecast latency, throughput or queue occupancy. Table 3 outlines this Reliability–Maintainability (RM) quality decomposition, mapping ISO/IEC sub-characteristics to architectural questions, underlying graph metrics, and targeted engineering remediations.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	**Fault Tolerance** (FT)	How broadly does failure propagate?	Reverse PageRank on $G^{\top}$, in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	**Modularity/Modifiability**	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 Automotive Safety Integrity Level [ASIL] ratings) and Security (which requires explicit threat models, such as STRIDE [Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege]) fall outside purely structural topology analysis and are reserved for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [47]:

1. **Fault Tolerance ($FT(v)$):** Measures error cascade potential on the transpose graph $G^{\top}_{\text{analysis}}$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \tag{14}$$

where $\mathrm{RPR}(v)$ is Reverse PageRank, $\mathrm{Deg}_{\mathrm{in}}(v)$ is normalized in-degree, and $\mathrm{CDPot}_{\mathrm{enh}}(v)$ is the enhanced Cascade Depth Potential term.

2. **Availability ($A(v)$):** Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component's own QoS weight:

$$A(v) = 0.2563 \cdot \mathrm{AP}_c^{\mathrm{dir}}(v) + 0.1998 \cdot \mathrm{QSPOF}(v) + 0.1998 \cdot \mathrm{BR}(v) + 0.2563 \cdot \mathrm{CDI}(v) + 0.0878 \cdot w(v) \quad (15)$$

where $\mathrm{AP}_c^{\mathrm{dir}}(v)$ is Directed Articulation Point severity, $\mathrm{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\mathrm{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\mathrm{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is the component's intrinsic QoS weight.

3. **Reliability ($R(v)$):** Blends Fault Tolerance and Availability hierarchically:

$$R(v) = r_\alpha \cdot FT(v) + (1 - r_\alpha) \cdot A(v), \quad r_\alpha = 0.36 \quad (16)$$

The blend weight is written r_α throughout to distinguish it from the payload-size coefficient α of Equation (3); the two are unrelated parameters and are screened separately (Supplementary Section S1). Intra-dimension pairwise comparison matrices are audited against Saaty's consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights apply a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports ranking sensitivity to λ).

4. **Maintainability ($M(v)$):** Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \mathrm{BT}(v) + 0.30 \cdot w_{\mathrm{out}}(v) + 0.15 \cdot \mathrm{CQP}(v) + 0.12 \cdot \mathrm{CouplingRisk}_{\mathrm{enh}}(v) + 0.08 \cdot (1 - \mathrm{CC}(v)) \quad (17)$$

where $\mathrm{BT}(v)$ is Betweenness Centrality, $w_{\mathrm{out}}(v)$ is QoS-weighted efferent coupling (out-degree), $\mathrm{CQP}(v)$ is the Code Quality Penalty, $\mathrm{CouplingRisk}_{\mathrm{enh}}(v)$ is an afferent/efferent coupling-risk imbalance term, and $\mathrm{CC}(v)$ is the local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (18)$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\mathrm{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\mathrm{static}}(v) \quad (19)$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \mathrm{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \mathrm{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This yields a directly actionable distinction: a component scoring high on A but low on FT is a single point of failure calling for horizontal replication, whereas one scoring high on FT is an error-cascade hub calling for circuit breakers, rate limiting and bulkhead isolation. Whether acting on these diagnoses improves any measured runtime property is not evaluated here (Section 8.4).

Table 4: Experimental evaluation corpus. The twelve synthetic topologies are the inductive Leave-One-Scenario-Out folds of Table 9; the five real-world systems are withheld from every training fold and used only for zero-shot transfer (Section 7.4). Counts are read from the committed topology files rather than from the generator configurations.

Dataset / Architecture	System Paradigm	$\|V\|$	$\|V_{\text{app}}\|$	Topics	Brokers	Hosts	Libs	$\|E\|$
Synthetic evaluation scenarios (`evaluation` role in the manifest)								
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	773
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,389
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	583
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	428
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	751
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,331
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Telecom RAN	5G Radio Access Network	225	120	55	8	20	22	858
Industrial SCADA	Plant Control Telemetry	254	140	70	4	25	15	818
Real-Time Gaming	Multiplayer State Sync	158	75	38	5	12	28	643
Logistics Fleet	Vehicle Telematics Mesh	205	110	50	7	18	20	792
Synthetic case study (LOSO fold; `case_study` role in the manifest)								
Air Traffic Management (ATM)	ICAO Global ATM Concept	74	26	27	5	8	8	271
Real-world reference systems (never used as training folds)								
Autoware.universe [73]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [74]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [75]	Real-World Microservices	90	41	30	3	8	8	162
Home Assistant [76]	Real-World Smart Home	63	24	22	3	6	8	119
EdgeX Foundry [77]	Real-World Industrial IoT	63	22	24	3	6	8	112
Synthetic subtotal (12 LOSO folds)		**2,461**	1,321	615	65	202	258	**11,317**
Real-world subtotal (5 systems)		**351**	141	120	16	32	42	**700**
Total		**2,812**						**12,017**

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 2,812 components across seventeen system architectures — twelve synthetic topologies that form the inductive cross-validation folds, and five real-world reference systems withheld entirely from training — as detailed in Table 4:

Here, $|V|$ is the sum of all five entity-type counts per scenario. The twelve synthetic topologies total 2,461 components; the eleven carrying the manifest's `evaluation` role account for 2,387 of these and the ATM case study for the remaining 74. The five real-world systems add 351. $|E|$ counts every raw structural relationship instance recorded in the scenario specification — the native substrate that simulation oracles traverse — rather than the derived `DEPENDS_ON` projection constructed for GNN training. All six structural relation types are included in that count (`PUBLISHES_TO`, `SUBSCRIBES_TO`, `ROUTES`, `RUNS_ON`, `CONNECTS_TO`, `USES`); we note the convention explicitly because the generator's own build log omits `CONNECTS_TO` and therefore reports slightly lower per-scenario totals. Every count in Table 4 is read directly from the committed topology files, whose byte-identical regeneration from configuration is asserted in continuous integration (Section 6.1.1).

Five real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The synthetic scenarios were produced by a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, minimum, maximum, Q_1, Q_3) for application publish and subscribe fan-out, applications per host, library fan-in, topic payload size, and categorical distributions over the three QoS dimensions. Graph degree distributions and clustering emerge directly from these parameters rather than being synthetically forced. Table 5 reports the generative parameters governing each topology's shape; complete configurations are included in the replication package.

Corpus Composition across Experimental Regimes. The experimental evaluation spans three complementary regimes with precisely bounded corpora:

1. *In-Distribution Evaluation (Table 7):* Evaluated on the seven core synthetic domains from Table 4 using stratified 60% train / 20% validation / 20% test node splits over five random seeds.

Table 5: Generative parameters of the eleven synthetic evaluation scenarios. Seed and fan-out figures are read from the committed configurations; per-entity counts are in Table 4. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config's declared QoS targets, which domain-driven assignment does not always realize (Section 6.1).

Scenario	Seed	Pub	Sub	Modal QoS (R/D/P)
Air Traffic Management (ATM)	42	1.2	2.9	RELIABLE/VOLATILE/HIGH (41–81%)
Autonomous Vehicle (AV)	1001	2.5	5.0	RELIABLE/TRANSIENT_LOCAL/HIGH (45–80%)
Enterprise Pub-Sub	7007	3.0	4.5	RELIABLE/TRANSIENT_LOCAL/MEDIUM (29–79%)
Financial Trading	3003	4.0	6.0	RELIABLE/PERSISTENT/HIGH (40–89%)
Healthcare Integration	4004	2.5	3.0	RELIABLE/PERSISTENT/HIGH (40–88%)
Hub-and-Spoke	5005	2.0	7.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (33–67%)
Industrial SCADA	1902	1.1	1.0	RELIABLE/TRANSIENT/HIGH (31–80%)
IoT Smart City	2002	2.0	1.5	BEST_EFFORT/VOLATILE/LOW (56–75%)
Logistics Fleet	2104	1.8	1.9	RELIABLE/TRANSIENT_LOCAL/MEDIUM (40–76%)
Microservices Mesh	6006	1.5	2.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (40–60%)
Real-Time Gaming	2003	2.6	2.8	BEST_EFFORT/VOLATILE/HIGH (39–71%)
Telecom RAN	1801	2.2	1.6	RELIABLE/VOLATILE/HIGH (36–67%)

2. *Inductive Leave-One-Scenario-Out (LOSO) Cross-Validation (Table 9):* Evaluated across twelve distinct inductive folds totaling 2,461 components: the seven core synthetic scenarios, four extended domain topologies (Telecom RAN, Industrial SCADA, Realtime Gaming, and Logistics Fleet) — 2,387 components between them — and an Air Traffic Management (ATM) network scenario contributing the remaining 74. In each fold, models are trained on eleven graphs and tested zero-shot on the held-out twelfth graph.
3. *Real-World Architectural Transfer (Tables 11 and 12):* The five open-source real-world systems (Autoware.universe, Cloud Microservices, Train-Ticket, Home Assistant, EdgeX Foundry) are never used as training folds; they are withheld entirely and used strictly for zero-shot architectural transfer validation.

Not every analysis in Section 7 runs on the full corpus: the sensitivity sweeps and the detection benchmark predate the four extended domains and operate on smaller cached subsets. Because a reader comparing figures across subsections would otherwise have no way to tell which population a number belongs to, Table 6 states the subset used by each analysis. Comparisons are made only within a row.

Table 6: Which corpus subset backs each analysis. Figures from different rows are not directly comparable, and we do not compare them.

Analysis	Scenario subset	n	Reported in
In-distribution ranking	Seven core synthetic domains	7	Tables 7–8
Inductive LOSO	Eleven evaluation scenarios + ATM	12	Table 9, Sections 7.1–7.2
QoS edge-feature ablation	Same twelve LOSO folds	12	Section 7.3.1
Weight sweeps and Morris screening	Six–seven core synthetic domains	6–7	Supplementary S1
Cross-oracle convergent validity	Seven core synthetic domains	7	Table 10
Anti-pattern detection, stratification	Seven core domains + ATM	8	Section 7.3.3
Relational attention illustration	ATM case study alone	1	Figure 3
Real-world zero-shot transfer	Five open-source systems	5	Tables 11–12

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than merely statically archived. Each dataset is deterministically generated from its configuration file via:

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset

regenerates *byte-identically* from its configuration and that all disk digests match the manifest. This guarantees that third parties can reproduce the exact graphs used in our experiments, rather than simply sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations drawn from three families. Predictor names state the family and the substrate: an `-N` suffix marks a model trained on the *native* multigraph, its absence the derived Application–Library flow projection, and a `-QoS` suffix marks a configuration that consumes declared QoS contracts. *SaG* throughout denotes the framework, never an individual predictor.

1. **Heterogeneous graph learning (typed HGT). HGT-QoS** (proposed): relation-specific Heterogeneous Graph Transformer (Section 4) ingesting the complete native multigraph with 16-dimensional continuous-categorical edge features that encode middleware QoS contracts. Its ablation **HGT**, which masks those QoS dimensions, is reported in Section 7.3.1.
2. **Homogeneous graph learning (untyped GAT). GAT-N-QoS**: homogeneous Graph Attention Network [56] trained on the identical native multigraph substrate with per-type input projections, but untyped, single-relation message passing. Its edge channel carries the scalar QoS aggregate $w(e)$ — dimension 0 of the same 16-D encoding HGT-QoS consumes — rather than the per-dimension decomposition; no homogeneous architecture in our suite ingests the full 16-D vector. The HGT-QoS–GAT-N-QoS contrast therefore bounds the *joint* contribution of relational typing and per-dimension QoS encoding. Section 7.3.1 separates the second factor within the typed architecture, and the corresponding unweighted ablation is **GAT-N**.
3. **Structural baselines (training-free). Topo-QoS**: QoS-weighted topological centrality evaluated on the derived application flow projection.
4. **Structural baselines (training-free). Topo**: structural centrality combining unweighted betweenness centrality and articulation point scoring on the flow projection.

In addition, the out-of-distribution evaluation (Table 9) reports **RM** ($Q(v)$, the deterministic hierarchical quality attribution model of Section 5) as a diagnostic reference baseline. RM is not fitted to rank failure impact; its inclusion demonstrates how much learned relational prediction adds over static structural attribution (Section 1.2). Furthermore, deterministic RM scoring drives every sensitivity sweep in Section 7.3, where closed-form formulations isolate parameter effects from neural training stochasticity.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate on matched substrates within their respective families:

- **Graph Learning Models (GAT-N-QoS, HGT-QoS):** Both learned neural predictors ingest the complete native typed multigraph across all five entity types in both in-distribution (Table 7) and out-of-distribution Leave-One-Scenario-Out (Table 9) evaluations — which is what the shared `-N` suffix records — so the comparison carries no multi-entity visibility confound. Node type reaches homogeneous GAT-N-QoS only through its per-type input projection layer, while message passing uses untyped GATConv with shared weights across all edges; in contrast, heterogeneous HGT-QoS employs relation-specific HGTConv weight matrices per edge triple alongside edge-type encodings. Substrate and node features are matched; the edge channel is not, since GAT-N-QoS consumes the scalar QoS aggregate $w(e)$ where HGT-QoS consumes all 16 dimensions (Section 6.2). Comparisons between the two therefore isolate relation-specific parameterization jointly with per-dimension QoS encoding, and we report the QoS factor separately in Section 7.3.1 rather than attributing the whole margin to typing.
- **Training-Free Structural Baselines (Topo, Topo-QoS):** Topological baselines are evaluated on the derived Application–Library `DEPENDS_ON` projection (Section 3.2). This projected substrate is necessary because in raw publish–subscribe multigraphs, Application nodes never route messages directly, resulting in near-zero betweenness and bridge ratios that yield degenerate, uninformative scores.

- **Ground-Truth Independence Guarantee:** Crucially, **no predictor in either group accesses** $G_{\textbf{structural}}$: that raw topology is strictly reserved as the substrate for ground-truth simulation oracles (Section 4.4), a guarantee formally verified by `tests/test_independence_guarantee.py`.

Regardless of substrate, all variants are scored on an identical, independently resolved Application node set (V_{app}, Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures in this paper use the high-contrast, colorblind-safe Okabe–Ito palette together with distinct marker, hatching and node-shape encodings, so that every distinction carried by colour is also carried by form and remains legible in monochrome.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall's rank correlation (τ) between predicted component rankings and ground-truth simulated impact $I^*(v)$ from the primary oracle (Section 4.3).

- **Critical-Set Identification:** Measured via $F_1@K$, Precision@K, and Recall@K for top-K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically as the top-K set overlap.

- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [78] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals over folds ($B = 2{,}000$) [79, 80], reported for every out-of-distribution predictor in Table 9 and for the paired contrasts in Section 7.1. Given the sample size, we regard those intervals rather than the p-values as the primary evidence. The unit of analysis is the scenario (in-distribution, $n = 7$) or the fold (LOSO, $n = 12$). The in-distribution design sits at the floor of its own resolving power — the smallest attainable two-sided p is 0.0156 at $n = 7$, so only a near-unanimous sign pattern can register at all. The twelve-fold LOSO design is materially better resourced: its floor is 0.00049, and it tolerates as many as four lost folds while still reaching $\alpha = 0.05$, provided those losses are the smallest in magnitude. Where a LOSO comparison nonetheless fails to reach significance (Section 7.1), the cause is therefore the size of the losing folds rather than the size of the corpus, and enlarging the corpus would not remedy it. We report p-values uncorrected and treat them as exploratory. Under a family-wise correction no in-distribution comparison in Table 8 can reach $\alpha/6 = 0.0083$ regardless of the data, and the unanimous 12/12 LOSO results survive a six-comparison correction ($\alpha/6 = 0.0083$) but not a seven-comparison one ($\alpha/7 = 0.0071$). Directional consistency across folds, rather than any single p-value, is what carries the out-of-distribution claims.

6.3.1. Evaluation Population

Every predictor within a given evaluation table is scored on an identical node population, resolved strictly from scenario topology and simulation ground truth — never from any model's predictions. Unless otherwise noted, this population is the **Application** set (V_{app}). This aligns with the framework's primary objective (forecasting application-layer cascading failures) and ensures a fair common denominator across both typed and untyped predictors. Pooling node types into a single global ranking conflates distinct base rates and impact distributions, shifting the resulting rank correlation outside the envelope of per-type correlations (Section 7.3). We therefore report stratified, single-population metrics throughout and explicitly identify any pooled figures.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits over five seeds $\{42, 123, 456, 789, 2024\}$. Each split is a deterministic function of node identity *and* seed, so a seed redraws the partition as well as the initialisation. The consequence for Table 7 is that dispersion across seeds is split noise on 10–60 held-out nodes for every variant — including the training-free baselines, whose scores are otherwise deterministic — and additionally training noise for the learned predictors. Per-seed standard deviations are reported in that table for this reason.

- **Inductive Leave-One-Scenario-Out (LOSO):** Models are trained on eleven scenarios and evaluated zero-shot on the held-out twelfth, testing zero-shot generalization across distinct architectural domains. Three protocol properties bear on the validity of the comparison, and all three apply identically to every variant. First, every learned variant receives the identical training set of $N - 1$ graphs: one scenario is designated the *primary* graph and the remainder are passed as additional inductive graphs. Second, message-passing depth is fixed at three layers for every fold rather than derived from the primary graph's size, so capacity does not vary with which scenario is held out. Third, early stopping and checkpoint selection are driven by a *held-out training scenario* — deterministically the median-sized inductive graph, excluded from the loss entirely — rather than by a within-scenario validation split of a training graph, since selecting on a split of a graph the model is already fitting selects for in-distribution fit under a protocol whose purpose is distribution shift. The outer holdout participates in none of these choices.
- **Real-World Architectural Transfer:** Evaluating models trained on synthetic corpora zero-shot on authentic open-source distributed systems without fine-tuning.

7. Results and Empirical Analysis

This section presents empirical results for RQ1–RQ5 across the twelve-fold inductive benchmark and five authentic open-source distributed systems. Evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 7 presents in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across seven representative distributed architecture domains.

Table 7: In-distribution held-out ρ against $I^*(v)$: mean over five seeds ± spread; n = held-out Application count. All four learned variants use the identical native multigraph, differing only in typing and edge channel (Section 6.2); topological baselines use the flow projection. Each seed redraws the 60/20/20 split as well as the initialization. Read with Table 8.

Scenario	n	Topo	Topo-QoS	GAT-N	GAT-N-QoS	HGT	HGT-QoS
AV System	16	0.362 ±0.169	0.762 ±0.159	0.632 ±0.224	0.758 ±0.110	**0.797** ±0.092	0.737 ±0.126
Enterprise	60	0.421 ±0.089	0.803 ±0.060	0.638 ±0.355	0.864 ±0.033	0.726 ±0.381	**0.903** ±0.014
Financial Trading	12	0.169 ±0.375	0.636 ±0.125	**0.864** ±0.047	0.573 ±0.483	0.758 ±0.189	0.590 ±0.245
Healthcare	10	−0.275 ±0.267	0.660 ±0.190	**0.863** ±0.039	0.802 ±0.068	0.475 ±0.387	0.478 ±0.327
Hub-and-Spoke	14	0.025 ±0.230	**0.545** ±0.260	0.538 ±0.219	0.441 ±0.308	0.367 ±0.571	0.425 ±0.462
IoT Smart City	40	0.124 ±0.129	0.361 ±0.082	0.790 ±0.103	0.639 ±0.191	0.825 ±0.053	**0.848** ±0.045
Microservices	18	0.337 ±0.202	**0.635** ±0.136	0.514 ±0.105	0.497 ±0.209	0.421 ±0.301	0.428 ±0.327
Mean	—	**0.166**	**0.629**	**0.691**	**0.653**	**0.624**	**0.630**

Table 8: Paired Wilcoxon signed-rank tests across in-distribution scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p-value	Significance
HGT-QoS vs. Topo	**+0.464**	7/7	0.0	**0.0156**	**Significant** ($p < 0.05$)
Topo-QoS vs. Topo	**+0.463**	7/7	0.0	**0.0156**	**Significant** ($p < 0.05$)
GAT-N vs. HGT-QoS	+0.061	4/7	9.0	0.469	Not significant
HGT-QoS vs. GAT-N-QoS	−0.023	3/7	12.0	0.813	Not significant
HGT vs. GAT-N	−0.067	3/7	8.0	0.375	Not significant
HGT-QoS vs. HGT	+0.006	5/7	11.0	0.688	Not significant
HGT-QoS vs. Topo-QoS	+0.001	2/7	10.0	0.578	Not significant
GAT-N-QoS vs. Topo-QoS	+0.024	3/7	13.0	0.938	Not significant

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Table 9: Inductive LOSO evaluation, Application population, twelve folds. Learned variants share training set, substrate, depth and selection rule (Section 6.3), differing only in typing and edge channel. Fold score = mean over five seeds; **Fold** σ = spread of the twelve fold means; **Seed** σ = median within-fold spread (zero by construction for deterministic baselines); CI = bootstrap over folds ($B = 2{,}000$).

Predictor / Reference	**Mean LOSO** ρ	**95% CI**	**Fold** σ	**Seed** σ	**Critical-Set** $F_1@K$	**Requires Training**
Training-free structural baselines						
Topo	0.250	$[0.128, 0.356]$	0.200	—	0.306	No
Topo-QoS	0.568	$[0.418, 0.686]$	0.237	—	0.353	No
Learned predictors (shared native substrate, matched training set)						
GAT-N	0.493	$[0.437, 0.541]$	**0.092**	0.158	0.417	Yes
GAT-N-QoS	0.581	$[0.493, 0.657]$	0.141	**0.017**	0.474	Yes
HGT	0.640	$[0.565, 0.716]$	0.138	0.069	0.466	Yes
HGT-QoS	**0.695**	$[0.631, 0.748]$	0.103	0.053	**0.507**	Yes
Diagnostic reference — not a ranking model						
RM / $Q(v)$	0.133	$[0.009, 0.247]$	0.215	—	0.258	No

Twelve LOSO folds are reported, comprising the eleven synthetic evaluation scenarios and the ATM case study, all specified in Table 4 (Section 6.1). In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively on the remaining eleven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the twelve folds, where the smallest attainable two-sided p is 0.00049. Per-fold evaluated populations range from 26 to 300 Application nodes, so $K = \text{round}(0.20\,|V_{\text{app}}|)$ ranges from 5 to 60. On $F_1@K$, HGT-QoS beats Topo-QoS in 8 of 12 folds ($\Delta = +0.154$, $W = 12.0$, $p = 0.034$) but separates from untyped GAT-N-QoS in 8 of 12 without reaching significance ($\Delta = +0.034$, $W = 25.0$, $p = 0.301$): critical-set identification distinguishes the typed learned model from the training-free baseline, but not from untyped learning.

Label-noise ceiling. These correlations are bounded by the reproducibility of the target they are scored against. Re-running the ground-truth oracle across the five seeds gives a test–retest rank correlation between 0.880 and 1.000 across the twelve folds (median 0.979; ten of twelve at or above 0.95), with Microservices the least reproducible at 0.880. HGT-QoS's $\rho = 0.695$ therefore recovers roughly 71% of the attainable signal against the median ceiling, and no predictor in Table 9 can exceed the reproducibility of its own labels. Top-K critical sets are the noisier construct by a wide margin: their cross-seed Jaccard has a median of 0.709 and falls to 0.400 (Microservices), 0.467 (Telecom RAN) and 0.481 (Logistics Fleet). That instability is the main reason the $F_1@K$ margins are less stable than the ranking margins, and it bounds how much weight any single critical-set comparison can carry. Notably, Microservices is both the least reproducible fold and one of the two on which typed learning loses (Section 7.2.1) — part of that deficit may be label noise rather than model failure.

Key Insights for RQ1:

1. **Typed learning is the best configuration, but not demonstrably better than the QoS baseline.** HGT-QoS leads all predictors out-of-distribution ($\rho = 0.695$), and the typed-vs-untyped margin excludes zero decisively (Section 7.2). Against training-free *Topo-QoS*, HGT-QoS is +0.127 (9/12, $W = 16.0$, $p = 0.077$, CI $[+0.011, +0.255]$) and HGT is +0.073 (10/12, $p = 0.129$, CI $[-0.038, +0.196]$). We decline to read the first as a win: the margin is carried almost entirely by ATM, where Topo-QoS fails outright ($\rho = -0.086$, its only negative fold) against HGT-QoS's 0.579. Excluding that fold the margin falls to +0.078 (8/11, $p = 0.148$). A bootstrap interval that excludes zero on the strength of one fold is not evidence of general superiority, and we report interval and sensitivity together rather than quoting whichever is more favourable.
2. **A QoS-weighted structural score is a genuinely strong baseline.** Topo-QoS reaches $\rho = 0.568$ zero-shot, beating unweighted Topo on 11 of 12 folds (+0.318, $p = 0.0010$) and remaining

indistinguishable from untyped learning in both directions (GAT-N trails by 0.075, $p = 0.077$; GAT-N-QoS leads by 0.013, $p = 0.850$). Any claim that graph learning is *required* must be made against this baseline, not against unweighted centrality.

3. **Power is not the limiting factor.** At $n = 12$ the design tolerates four lost folds and still reaches $\alpha = 0.05$, provided the losses are smallest in magnitude. HGT-QoS's are not: it loses Enterprise (-0.268), Microservices (-0.080) and Real-Time Gaming (-0.004), and Enterprise is the largest $|\Delta|$ among them — which is what holds W at 16.0. Enlarging the corpus will not resolve this; the two substantive inversions must be understood instead (Section 7.2.1).
4. **The explanation layer is weakly predictive, not noise.** RM/$Q(v)$ reaches $\rho = 0.133$, losing to unweighted Topo on all twelve folds (-0.117, $p = 0.0005$), so no ranking claim is made for it. Its interval $[0.009, 0.247]$ stays above zero and it supplies interpretable diagnostics without training (Section 5). It appears in Table 9 as a reference point, not a competitor.

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the specific contribution of node and edge typing, we contrast the relation-specific Heterogeneous Graph Transformer against the homogeneous Graph Attention Network on the shared native multigraph substrate, with the identical training set, depth and model-selection rule:

- **In-Distribution Fitting (Table 7): typing does not help.** On familiar architectures, typed message passing carries no advantage at all. HGT-QoS reaches $\rho = 0.630$ against GAT-N-QoS's 0.653 ($\Delta\rho = -0.023$, won in 3 of 7, $W = 12.0$, $p = 0.813$), and the unweighted pair runs the same way (HGT 0.624 vs. GAT-N 0.691, -0.067, $p = 0.375$). The best in-distribution mean in the table belongs to *untyped* GAT-N. None of these differences is significant at $n = 7$, and per-seed dispersion is large (median within-scenario σ of 0.105–0.301 across the learned variants, reaching 0.571 on Hub-and-Spoke), so the honest reading is parity rather than a reversal — but there is no in-distribution typing benefit to report.

- **Out-of-Distribution Generalization (LOSO, Table 9):** Under inductive distribution shift, the typed advantage is the most robust effect in this study. HGT-QoS outperforms GAT-N-QoS by +0.114 ($\rho = 0.695$ vs. 0.581), winning 11 of 12 folds ($W = 8.0$, $p = 0.0122$; 95% CI $[+0.048, +0.170]$). The unweighted pair replicates it and slightly exceeds it: HGT over GAT-N by +0.147, 11 of 12 folds, $W = 1.0$, $p = 0.0010$, CI $[+0.101, +0.185]$.

Typing is an inductive bias, not a capacity advantage. The contrast between the two regimes is the finding. Given training data from the same generator as the test split, untyped message passing matches typed and nominally exceeds it. Asked to transfer to an unseen architecture, the typed model wins 11 folds of 12 by +0.114, while the untyped model loses 0.088 of its in-distribution standing ($0.653 \rightarrow 0.581$) and the typed model gains 0.065 ($0.630 \rightarrow 0.695$). The reading is that relation-specific parameters add no fitting capacity — with enough same-distribution data an untyped attention mechanism recovers the same structure — but encode which distinctions survive a change of topology. Distinguishing `PUBLISHES_TO` dissemination from `RUNS_ON` placement is a constraint rather than extra expressiveness, and constraints pay off exactly where distribution shift would otherwise mislead. This is a more specific claim than we set out to test, and a more useful one for a pre-deployment gate, where every architecture analysed is by construction new.

The single fold typed learning loses is the same in both contrasts: ATM, the smallest graph in the corpus (26 Applications) and the only one designated a case study. Excluding it, both contrasts win every remaining fold (+0.139 and +0.161, 11/11, $p = 0.0010$). We report the twelve-fold figure as the headline regardless, because excluding an inconvenient fold to recover unanimity is precisely the move a reader should distrust.

Methodological Controls and Substrate Parity. To ensure that the typed–untyped margin reflects genuine architectural inductive biases rather than experimental artifacts, all learned predictors in Table 9 operate under strict substrate and training-set parity: every model receives all $N - 1$ training graphs, message-passing depth is fixed at three layers, and checkpoint selection is governed by held-out scenario validation rather than within-graph splits. Under these controlled conditions, homogeneous GAT-N reaches $\rho = 0.493$, so

the observed heterogeneous advantage ($\Delta\rho = +0.114$, won in 11 of 12 folds, $p = 0.0122$) is attributable to relational typing rather than to substrate, training set, depth or selection rule.

7.2.1. Where Typed Learning Fails, and How It Can Be Detected

HGT-QoS loses to Topo-QoS on three folds, but only two of them are substantive: Enterprise ($\rho = 0.569$ vs. 0.838) and Microservices (0.483 vs. 0.563). The third, Real-Time Gaming (0.776 vs. 0.780), is a tie to within 0.004 and carries no interpretive weight. It is the two substantive inversions that hold the RQ1 comparison below significance, and both are folds on which the training-free baseline is unusually strong, which suggests the model is discarding structural signal the baseline retains rather than failing to learn.

A candidate label-free signature, and why we do not claim it. Let $\hat{\sigma}$ denote the standard deviation of the model's own predicted scores over the held-out Application population — a quantity computable at inference time, without labels. The intuition is that a model whose output collapses toward a constant on an unfamiliar architecture is signalling its own unreliability. Two observations are consistent with it: Enterprise carries the lowest $\hat{\sigma}$ in the corpus (0.111 against a median of 0.173) and Microservices the third lowest (0.138), and those are the two folds HGT-QoS loses.

The correlation does not survive inspection. Across the twelve folds, $\hat{\sigma}$ and the margin over Topo-QoS correlate at $\rho_s = -0.126$ ($p = 0.697$) for HGT-QoS — the wrong sign — and $+0.245$ ($p = 0.443$) for HGT. The only fold set on which the relationship is significant belongs to the untyped baseline (GAT-N, $\rho_s = +0.706$, $p = 0.010$), which is not the model we propose. The counterexample is Healthcare: it carries the second-lowest $\hat{\sigma}$ in the corpus (0.123) and one of the largest positive margins ($+0.284$). A low-dispersion prediction is therefore not a reliable warning; on this corpus it is associated with the two worst folds and one of the best.

We report this negative result rather than the two-fold coincidence that motivated it. A label-free confidence signal for pre-deployment gating would be a genuinely valuable property, and we had one in an earlier analysis, but it does not replicate on the present corpus and we do not build on it. The zero-shot dispersion figures of Section 7.4.1 must be read against this: whatever $\hat{\sigma}$ tracks across five real systems, it does not track fold difficulty across twelve synthetic ones.

Graph size does not explain the failure either. The rank correlation between evaluated fold size and margin is -0.357 ($p = 0.255$) — weakly negative, but not significant and not consistent in sign across the fold set: the three largest wins are the smallest graph in the corpus (ATM, 26 Applications, $+0.665$), the second largest (IoT Smart City, 200, $+0.332$) and the second smallest (Healthcare, 50, $+0.284$). Enterprise is the largest fold and one of the two losses, but Microservices is mid-sized, so size alone orders neither the wins nor the losses.

Feature *scale* is a different matter from graph size, and the cross-scenario scale drift documented in `results/feature_shift_diagnostic.md` remains the most plausible account of the Enterprise deficit: it is the corpus's largest graph, and within-graph feature rank normalisation is the transform whose effect should be largest there. We offer this as an explanation consistent with the evidence rather than a tested one: the configuration reported in Table 9 is fixed across folds by design (Section 6.3), so this study does not isolate the contribution of that transform on the Enterprise fold.

What we cannot offer is a way to tell in advance which architectures these are. The $\hat{\sigma}$ signature above does not provide it, and no other label-free indicator we examined — graph size, edge density, entity-type mix — orders the folds either. A practitioner applying the model to an unseen architecture therefore has no reliable warning that it is one of the two on which a training-free baseline would have served better. We regard this, rather than the mean correlation, as the most consequential open limitation of the predictive pathway for deployment as a gate (Section 8.4).

7.3. RQ3: Ablations and Sensitivity Analysis

This section reports the ablations that bear on a headline claim — the QoS edge encoding, cross-oracle agreement, and the per-type stratification that governs how every other result in this paper is read. The parameter-sensitivity sweeps over the explanation layer's ten declared weight constants establish robustness rather than any finding of their own, and are reported in full in the supplementary material (Supplementary

Sections S1–S2). Their collective result is stated here so the body remains self-contained: of the ten constants, only the Fault-Tolerance/Availability blend r_α and the AHP shrinkage λ carry appreciable influence on ρ ($\mu^* = 0.144$ and 0.117 under Morris screening, against ≤ 0.023 for the remaining eight), and no setting of the topic-weight or QoS sub-weight constants would change any comparison reported above. The elicited AHP weights are, notably, *anti*-predictive: rank correlation falls monotonically from 0.262 under a uniform prior to 0.166 under raw AHP judgement. We retain them because RM is an attribution instrument rather than a ranking model, and discuss that trade in Section 8.4.

7.3.1. QoS Feature Ablation

To isolate the specific empirical contribution of the continuous-categorical QoS edge features (Section 4.1.1), we evaluated **HGT**, an un-augmented ablation of HGT-QoS whose edge features contain only scalar coupling and relation one-hot encodings.

Under the inductive LOSO evaluation, the QoS edge encodings carry a clear ranking benefit in both architectures. Mean LOSO rank correlation is $\rho = 0.695$ with the encodings and 0.640 without them ($\Delta\rho = +0.054$, won in 11 of 12 folds, $W = 7.0$, $p = 0.0093$; 95% bootstrap CI $[+0.021, +0.091]$). The homogeneous architecture agrees in direction and magnitude: GAT-N-QoS achieves 0.581 against GAT-N 0.493 ($\Delta\rho = +0.088$, won in 9 of 12 folds, $W = 12.0$, $p = 0.034$, CI $[+0.021, +0.151]$). The typed result is robust to dropping the ATM fold (+0.049, 10 of 11, $p = 0.019$); the homogeneous one is not (+0.071, 8 of 11, $p = 0.067$), so we treat the typed evidence as the finding and the homogeneous replication as supporting.

The encodings also improve optimisation reproducibility, though less symmetrically than the ranking result. The median within-fold standard deviation across five random seeds is 0.053 for HGT-QoS against 0.069 for un-augmented HGT, and 0.017 for GAT-N-QoS against 0.158 for GAT-N — a large effect in the homogeneous pair and a modest one in the typed pair. We read the ranking gain as the primary result and the variance reduction as a secondary property, rather than the reverse.

A note on what makes this ablation informative. An earlier version of the generator emitted a near-constant QoS profile in most scenarios, which left the QoS channel with almost no variance to encode and produced a null result here. The corpus reported in this paper does not have that property: modal QoS shares range from 29% to 89% across the twelve scenarios (Table 5), so every fold carries genuine variation in declared reliability, durability and priority. The ablation is meaningful only under that condition, and we state it because a QoS ablation run on a QoS-degenerate corpus measures nothing. As noted in Section 4.1.1, three schema dimensions (`has_deadline`, `deadline_ns_log`, `max_blocking_ms_log`) remain zero throughout the corpus and are reserved extension points; the gains reported here come from the four active dimensions alone.

7.3.2. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (`FaultInjector`), $I_{\text{comp}}(v)$ (`FailureSimulator`), and $I_{\text{dyn}}(v)$ (`MessageFlowSimulator`) over seven scenarios, summarized in Table 10:

Table 10: Inter-oracle agreement across simulation paradigms (chance top-K Jaccard is 0.111) over 7 diverse benchmark topologies. I_{dyn} denotes the queue-flow discrete-event simulation oracle, I^* denotes the graph topological cascade injection oracle, and I_{comp} denotes the multi-criteria composite oracle. The negative lower bounds on I_{comp} reflect a single scenario (`hub_and_spoke`) driven by tied zero-inflation rather than directional inversion (Supplementary Section S2).

Oracle pair	**Mean ρ (range)**	**Mean τ**	**Jaccard@K**	**Tie-robust**
I_{dyn} vs. I^*	**0.907** (0.748–0.985)	0.788	0.486	0.509
I_{comp} vs. I^*	0.425 (−0.044–0.654)	0.311	0.240	0.258
I_{comp} vs. I_{dyn}	0.427 (−0.037–0.654)	0.312	0.276	0.275

Ordering converges strongly between the queue-flow simulator and topological cascade oracle ($\rho = 0.907$). Top-K set agreement is more conservative (Jaccard 0.486), bounded by discrete threshold sensitivity in non-linear cascades.

7.3.3. Anti-Pattern Detection and Node-Type Stratification

Validated against $I_{\text{comp}}(v)$, the rule-based anti-pattern catalog reaches mean precision 0.236 and recall 0.887 ($F_1 = 0.372$) — but it implicates 94.3% of scored components, so that recall is close to what indiscriminate flagging achieves and the precision is near the base rate. The ATM scaling sweep (29 to 444 components, five seeds, seed-invariant) shows the mechanism: Cohen's κ declines monotonically from 0.074 to 0.019 as the flag rate rises from 86.7% to 92.3%. The catalog does not become wrong at scale so much as it stops discriminating. We report it as a characterisation of the catalog rather than a working triage mechanism, and delegate critical-set identification to the continuous rankers of Sections 7.1–7.2. A nineteenth detector, `DEEP_PIPELINE`, is excluded for path-combinatorial explosion (247,761 paths on a 29-component fixture).

Stratification vs. Pooling: Measured against the composite oracle $I_{\text{comp}}(v)$ over the eight scenarios of the detection benchmark, stratified RM rank correlations are $\rho = 0.515$ (Application, 8 scenarios), 0.183 (Broker, 6 scenarios), and 0.149 (Node, 8 scenarios), while pooled correlation across all types collapses to $\rho = 0.057$. Pooling node types triggers Simpson's paradox by conflating disparate structural base rates across architectural layers: the pooled figure sits below every per-type figure it aggregates. This is why every evaluation in this paper is reported on a single stratum, and why pooled critical-set numbers should be read as inflated wherever they appear (Section 7.4). On that same pooled benchmark, unweighted degree centrality attains $\rho = 0.161$ and $F_1 = 0.451$ against RM's 0.057 and 0.321 — RM is beaten by degree on both, confirming that it serves as an attribution instrument rather than an unconstrained global ranker.

7.3.4. HGT Attention Weight Analysis

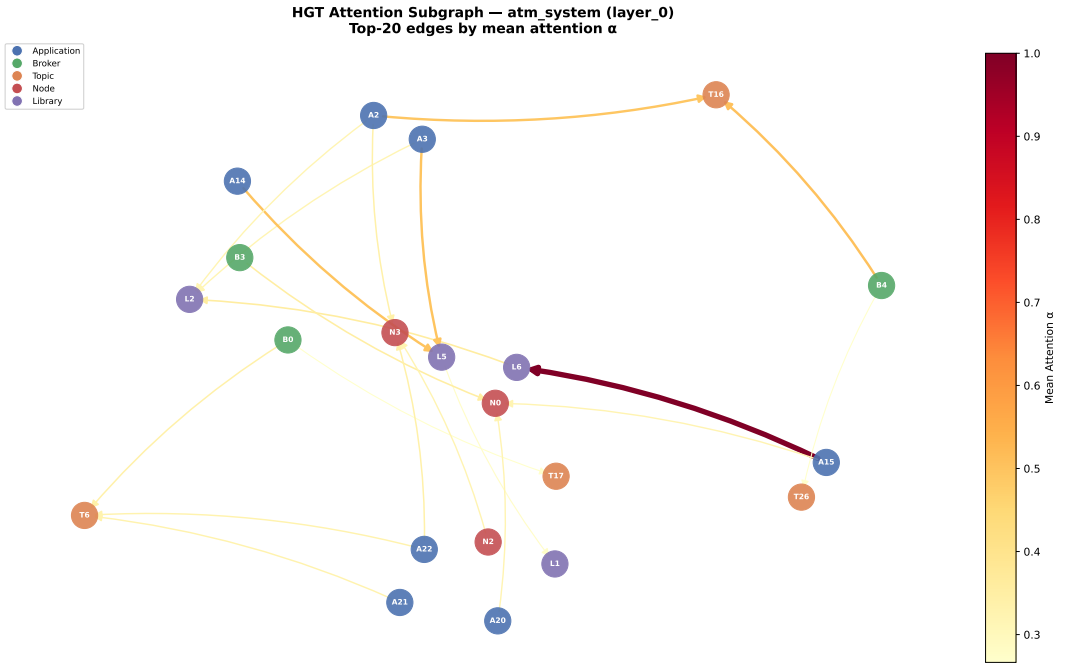

Figure 3: Relational attention extracted from the trained HGT on the ATM case study. Peak attention focuses on `USES` (Application → Library) and `ROUTES` (Broker → Topic) channels.

Aggregated by relation type over the ATM case study, first-layer mean attention orders `USES` into libraries (0.227, 0.215) above `ROUTES` (0.194), `SUBSCRIBES_TO` (0.176), `PUBLISHES_TO` (0.163) and `RUNS_ON` (0.153), with the same ordering to within 0.02 in layers two and three. Two cautions bound what this supports. The spread across all eight relation types is narrow (0.15–0.23), so this is a tendency, not a separation; and because α is a softmax over each destination's incoming edges, mean α is driven substantially by in-degree —

the top-ranked `Library→Library` relation carries 4 edges, and the largest weight in the graph ($\alpha_{uv} = 1.00$) sits at a destination of in-degree one, where $\alpha = 1$ holds by construction. Figure 3 is therefore a qualitative illustration on one topology at one seed without a significance test, not evidence for why typing helps.

7.4. RQ4: Real-World Distributed Architecture Validation

We evaluated the framework on five open-source distributed systems transcribed from their public repositories by dedicated architectural adapters, with results presented in Table 11. We note the scope of this evaluation before reporting it. Online Boutique is a vendor demonstration application and Train-Ticket an academic benchmark, Home Assistant is an authentic open-source IoT automation platform, and EdgeX Foundry is an industrial edge computing reference implementation; each is an abstraction of its upstream repository rather than a complete transcription; and their labels come from the same simulation oracle used throughout, so what follows tests topological generalization, not agreement with observed field failures. Two further limits bound what this table can support. First, the predictor scored in Table 11 is the deterministic explanation layer $Q(v)$ together with the two training-free topological baselines, and it is scored against $I_{\text{comp}}(v)$; the learned model is evaluated separately in Section 7.4.1 against $I^*(v)$, and the two are not commensurable. Second, the labels are strongly tied at zero in several architectures — 13 of 32 Applications in Autoware, 14 of 22 in Cloud Microservices, 27 of 41 in Train-Ticket, and 12 of 22 in EdgeX carry zero simulated impact (the complement of the "Impactful Apps" column), whereas Home Assistant exhibits much lower zero-inflation with 17 of 24 applications actively participating in failure cascades. Spearman ρ over populations that are heavily tied is dominated by how those ties are handled; the values below use midrank ties throughout.

Table 11: Open-source reference systems scored by the deterministic explanation layer RM / $Q(v)$ — no GNN checkpoint is invoked, so $\pm$ is oracle variance over five simulation seeds, not training variance. "Gain vs. Deg" is $|\rho_Q| - |\rho_{\text{deg}}|$. Ground truth here is $I_{\text{comp}}(v)$, whereas Table 12 uses $I^*(v)$: **the two must not be read against each other**. Simulator-derived labels, not observed incidents.

Real-World Architecture	$\lvert V\rvert$	$\lvert V_{\text{app}}\rvert$	Spearman ρ	Kendall τ	App $F_1@K$	Pooled $F_1@K$	Impactful Apps	Gain vs. Deg
Autoware.universe (ROS 2)	75	32	**0.685 ± 0.010**	0.513	0.333	0.800	19 / 32	+0.357
Cloud Microservices Mesh	60	22	**0.778 ± 0.001**	0.639	0.500	1.000	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	**0.759 ± 0.001**	0.605	0.625	1.000	14 / 41	+0.264
Home Assistant (Smart Home)	63	24	**0.514 ± 0.016**	0.373	0.250	0.667	17 / 24	+0.289
EdgeX Foundry (Industrial IoT)	63	22	**0.800 ± 0.015**	0.563	0.000	1.000	10 / 22	+0.427

Key Insights for RQ4:

1. **The explanation layer ranks unseen architectures well, but this is not a learned-transfer result.** RM / $Q(v)$ attains $\rho = 0.800$ on EdgeX Foundry, 0.778 on Cloud Microservices, 0.759 on Train-Ticket, 0.685 on Autoware.universe, and 0.514 on Home Assistant. Because $Q(v)$ is a closed-form scoring function rather than a fitted model, applying it to a new architecture involves no transfer in the inductive sense of Section 7.2: nothing was trained, and no distribution shift is being survived. What this table establishes is that the deterministic attribution of Section 5 remains informative on architectures we did not generate. The learned model is a separate question, answered in Section 7.4.1. Table 11 is not commensurable with Table 9's RM column ($\rho = 0.133$), and the gap has two separate causes that should not be conflated. The larger is the oracle: Table 11 scores $Q(v)$ against $I_{\text{comp}}(v)$ (`FailureSimulator`), whereas Table 9 scores it against $I^*(v)$ (`FaultInjector`), and the two agree at only $\rho = 0.425$ (Section 7.3). Scored against $I^*(v)$ on these same five systems, RM attains $\rho = 0.516$ (Table 12) — still far above its synthetic-corpus figure, so the remainder is a real-versus-generated difference, with the tied-label structure described above contributing to both. The practical consequence is that no RM figure in this paper should be compared across tables without checking which oracle produced it.
2. **Stratified vs. Pooled Critical Set Identification:** On the stratified Application population (V_{app} with $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$), $Q(v)$ identifies the top-20% critical services with $F_1@K = 0.333$ in Autoware, 0.500 in Cloud Microservices, 0.625 in Train-Ticket, 0.250 in Home Assistant, and 0.000 in EdgeX Foundry (where failure impact is dispersed broadly across active device services). When

evaluated across the pooled multigraph ($K = \text{round}(0.20 \cdot |V|)$), the pooled $F_1@K$ reaches 0.800, 1.000, 1.000, 0.667, and 1.000, respectively. This disparity is the base-rate phenomenon of Section 6.3: the passive infrastructure entities (Topics, Nodes, Libraries) carry constant zero failure impact under the injector configuration used for these runs, so a pooled top-K set is largely populated by correctly identifying inert nodes. The pooled column is therefore inflated and we do not read it; the stratified V_{app} column is the operational figure.

3. **Framework Acceptance Gates and Cross-Domain Disparities:** The framework ships a five-condition release gate ($\rho \geq 0.75$, $F_1 \geq 0.65$, SPOF $F_1 \geq 0.60$, fault-tolerance ratio ≤ 0.30, prediction gain ≥ 0.02). EdgeX Foundry passes all five gates across all five seeds (100% pass rate), driven by strong rank correlation ($\rho = 0.800$), pooled $F_1 = 1.00$, and robust articulation detection (SPOF $F_1 = 0.80$). In contrast, the pass rate is zero on the other four architectures: Autoware fails the ρ and SPOF conditions, Cloud Microservices fails SPOF and prediction gain, Train-Ticket fails SPOF, and Home Assistant misses the ρ gate ($\rho = 0.514$) despite scoring a perfect SPOF $F_1 = 1.00$. This indicates that while structural rank ordering transfers well across distributed paradigms, absolute gating thresholds require domain-specific calibration.
4. **Predictive Advantage over Structural Heuristics:** The "Gain vs. Deg" column reports $Q(v)$'s rank-correlation margin over an unweighted degree centrality baseline (+0.427 on EdgeX Foundry, +0.357 on Autoware, +0.289 on Home Assistant, +0.264 on Train-Ticket, +0.014 on Cloud Microservices), demonstrating that hierarchical attribution (Section 5) consistently resolves topological fragility that degree centrality obscures across diverse paradigms. As before, Wilcoxon signed-rank tests compare errors across distinct scales and do not reach significance ($p \geq 0.33$), but we report them transparently.
5. **The QoS-weighted heuristic does not replicate its synthetic-corpus advantage here.** Both training-free baselines are computable on these systems — every topic in all five adapters carries declared `durability`, `reliability` and `transport_priority` profiles, and every derived `DEPENDS_ON` edge consequently carries a non-unit QoS weight. Scored on the identical Application population and ground truth, `Topo` attains $\rho = 0.346/0.928/0.588$ and `Topo-QoS` 0.362/0.884/0.522 on Autoware, Cloud Microservices and Train-Ticket. QoS weighting does not universally improve ranking on non-generated architectures, consistent with the low-$w(t)$-variance caveat of Section 6.2. Projection-based baselines assign constant zero to Applications carrying no derived `DEPENDS_ON` edge, accentuating sensitivity to tied ground truth.

7.4.1. Zero-Shot Transfer of the Learned Model

The typing result of Section 7.2 was established on scenarios from our parameterized generator. To test whether graph learning transfers zero-shot to architectures we did not author, we trained HGT-QoS on all twelve synthetic scenarios and evaluated it on the five open-source systems against $I^*(v)$ (`FaultInjector`) across five seeds ($\{42, 123, 456, 789, 2024\}$).

Configuration, stated in full. This evaluation does not use the configuration of Table 9, and the difference must be on the record before the numbers are read. It employs within-graph rank normalization of node features and labels, a depth of two message-passing layers rather than three, and a budget of 150 epochs rather than 300. The motivation is the cross-scenario feature-scale drift documented in `results/feature_shift_diagnostic.md`: the five transcribed architectures differ in scale from the generated corpus more sharply than the generated scenarios differ from each other, and unnormalised features transfer poorly across that gap.

That motivation is a reason to expect the transform to help; it is not evidence that the specific depth and epoch budget were fixed independently of the results they produce. We therefore make no zero-shot-purity claim for the numbers in Table 12 beyond the literal one: no real-world system contributed a training gradient, and none was used for early stopping or checkpoint selection. Whether the configuration itself was chosen with knowledge of these five systems is a question the released harness answers and the reader should check; we flag it rather than let the phrase "zero-shot" carry more weight than it can bear. Table 12 presents the comparison against the training-free references scored on the identical population and oracle.

Key Insights for Real-World Transfer:

Table 12: Zero-shot transfer to five open-source systems, scored against $I^*(v)$ on the Application population. HGT-QoS trains on all twelve synthetic scenarios ($\pm$ = spread over five seeds); RM and Topo are training-free, scored on identical labels and nodes. $\rho_{>0}$ **restricts the correlation to the** $n_{>0}$ **components that actually propagate a failure and is the column to read for ranking quality**; full-population ρ conflates that with separating active from inert. `Topo-QoS` is absent (Section 8.4).

Real-World Architecture	$\lvert V_{\mathbf{app}}\rvert$	**RM** ρ	**Topo** ρ	**HGT-QoS** ρ	**HGT-QoS** $\rho_{>0}$	$n_{>0}$	$F_1@K$
Cloud Microservices Mesh	22	0.777	**0.891**	0.492 $\pm$0.203	**−0.249**	18	0.300
Train-Ticket Booking Mesh	41	0.713	0.528	**0.695** $\pm$0.078	**−0.237**	22	0.375
Autoware.universe (ROS 2)	32	0.357	0.307	**0.717** $\pm$0.072	+0.549	28	0.600
EdgeX Foundry (Industrial IoT)	22	0.470	0.534	**0.690** $\pm$0.083	+0.240	19	0.500
Home Assistant (Smart Home)	24	0.265	0.297	**0.817** $\pm$0.015	+0.596	23	0.520
Mean	—	0.516	0.511	**0.682**	+0.180	—	0.459

1. **The full-population figure is not a ranking result.** On the whole Application population HGT-QoS reaches $\rho = 0.682$ against 0.511 for Topo and 0.516 for RM, leading on four of five systems. Read alone, that looks like successful zero-shot transfer. It is not, because between 18% and 59% of Applications in these architectures carry exactly zero simulated impact, and a correlation computed over a population that is heavily tied at zero rewards separating the inert from the active at least as much as ordering the active correctly.
2. **Restricted to components that actually propagate failures, transfer is not established.** On the active stratum the mean falls from 0.682 to +0.180, and two of the five systems invert: Cloud Microservices to $\rho_{>0} = -0.249$ ($n = 18$) and Train-Ticket to -0.237 ($n = 22$). Both are the microservice call-tree architectures. The three pub-sub systems hold up (+0.549 Autoware, +0.596 Home Assistant, +0.240 EdgeX), so the pattern is architectural rather than random — but a method that anti-correlates with the truth on two of five held-out systems has not demonstrated zero-shot generalisation, and we do not claim it. **We therefore report RQ4 as a negative result:** learned relational transfer to authentic open-source architectures is not established by this evidence.
3. **What the full-population number does support.** Separating components that propagate failures from those that do not is itself the operationally useful half of the task — a gate that correctly identifies which two thirds of a system cannot cause a cascade has narrowed the review surface, even if it orders the remainder poorly. The $F_1@K$ column (0.459 mean) is the honest expression of that capability, and it is what a practitioner would act on. We separate the two claims rather than let the first stand in for the second.
4. **Why the microservice architectures invert.** Both are deep synchronous RPC call trees rather than pub-sub meshes: a large fraction of services are terminal sinks whose failure reaches nobody, and among the minority that do propagate, impact is governed by position in a call hierarchy that the model, trained entirely on generated pub-sub topologies, has never seen. Topo captures the upstream bottleneck structure directly and leads on Cloud Microservices ($\rho = 0.891$). This is a domain-shift limit of the training corpus, not a defect of typed message passing, and it marks the boundary of what our synthetic corpus can prepare a model for.

The $\hat{\sigma}$ dispersion values that an earlier version of this analysis used to flag the Cloud Microservices failure are not reported here as a diagnostic: the same quantity fails to track fold difficulty on the twelve-fold synthetic corpus, where it carries the wrong sign for this model (Section 7.2.1). A correlation of $\rho_s = +0.600$ over five systems — which cannot reach significance at $n = 5$ — is not sufficient to reinstate it.

7.5. RQ5: Analysis Cost and Its Comparison Against Simulation

RQ5 quantifies computational overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 13:

The neural model is the cheapest stage, and the deterministic one is expensive. At 2,000 components the HGT forward pass takes 56 ms against 239 s for deterministic structural analysis — a ratio of 4,259×. That ratio locates cost *inside* the pipeline; it is not the cost of evaluating an architecture. Indices 0–17 of every node feature vector (Section 3.4) — betweenness, closeness, reverse PageRank, articulation

Table 13: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$\|V\|$	$\|E\|$	**Analyse (s)**	**Graph→tensor (s)**	**HGT forward (ms)**	**Analyse : forward**
249	1,127	1.74	0.010	26.5	66×
499	2,402	8.32	0.022	16.4	509×
999	6,422	44.54	0.056	21.1	2,108×
1,998	19,301	239.34	0.157	56.2	**4,259**×

and bridge scores — are products of that same analysis stage, so the forward pass cannot run without it. End-to-end evaluation of an unseen 2,000-component architecture costs about four minutes, of which the learned model is 0.02%; the 56 ms figure is the marginal cost of re-scoring an already-analysed graph. Across the eight-scenario detection benchmark the complete gate (structural analysis plus 18 anti-pattern detectors) runs in 0.04–82.7 s, the upper bound being the 520-component Enterprise mesh.

Cost is dominated by one metric, and it grew. Measured cost now tracks the stage's $O(|V|^2 + |V||E|)$ bound closely: from 249 to 1,998 components wall-clock rises 138× against a 137× growth in $|V||E|$. The dominant term is the Connectivity Degradation Index, which is computed for every node in the main connected component rather than for articulation points alone. That choice is deliberate and is a correctness requirement rather than an oversight: gating CDI to articulation points leaves it identically zero for every node whose removal does not literally disconnect the graph, which drives $A(v)$ to a near-constant in the redundant multi-publisher topologies this framework targets. The cost is the price of a non-degenerate Availability score, and we report it rather than the cheaper gated variant we could have measured.

7.5.1. The Gate Is Not Cheaper Than the Simulation It Replaces

The framing that motivated this analysis — static gating as a low-cost substitute for dynamic simulation — does not survive measurement against our own oracle. Timing the `FaultInjector` labelling sweep (five seeds, the full ground-truth run) on the same corpus and the same idle hardware gives 2.5–6.3 s per scenario, against 0.04–82.7 s for the analysis gate. On the largest scenario the comparison is 6.3 s of simulation against 82.7 s of static analysis: **the gate costs roughly thirteen times more than the simulation it is meant to displace.**

Two things follow, and we state both. First, no claim in this paper that static analysis is computationally cheaper than simulation is supported, and we withdraw it. Our cascade oracle is an in-process breadth-first traversal, and traversing a graph is simply cheaper than computing per-node connectivity degradation over it. Second, the argument that survives is narrower and concerns *what* the two require rather than what they cost: the oracle needs a topology with declared failure semantics and produces labels for components it can express, whereas chaos engineering and hardware-in-the-loop validation — the practices a pre-deployment gate would actually displace in industry — need provisioned clusters, carry operational risk, and run at cluster-hour scale. That comparison is plausible and is the one made in Section 2.1, but we have not measured it, and it should not be read as established here.

The quantity measured throughout is wall-clock time on a single CPU core. We deliberately do not convert it into an energy or carbon figure (Section 8.2). What the measurements support is the narrow claim that pre-deployment analysis of a few-hundred-component architecture fits inside a pull-request budget without provisioning any runtime infrastructure. They do not support a claim of general computational efficiency, and at the scales invoked in Section 1.1 — an order of magnitude beyond what we measured, against an $O(|V|^2 + |V||E|)$ stage — the present implementation would not fit that budget at all.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

When to use Topo-QoS, and when to use HGT-QoS. The results do not support a simple recommendation of the learned model over the closed-form one, and we set out the trade-off as measured rather than as hoped.

1. **Training-free ranking (`Topo-QoS`).** It requires no training, no checkpoint storage and no retraining as a corpus evolves, reaching $\rho = 0.568$ zero-shot across twelve unseen synthetic architectures and ≈ 0.51 across five open-source systems. Nothing in this study establishes that a learned model beats it on ranking: the margin is $+0.127$ ($p = 0.077$) and collapses to $+0.078$ when one fold is removed (Section 7.1). For a team that wants a criticality ordering and nothing more, this is the defensible default, and we say so despite proposing the alternative.
2. **Learned relational prediction (`HGT-QoS`).** Its established advantage is over *untyped* learning under distribution shift ($+0.114$, 11 of 12 folds), not over closed-form centrality. It also leads on critical-set identification, where it does separate from the training-free baseline ($F_1@K$ $+0.154$, $p = 0.034$). Two further capabilities have no closed-form counterpart, and they are the substantive reason to prefer it where the extra machinery is affordable:
   - *Typed relational attention* exposes *which* channels mediate a cascade, rather than only which components rank highly. Figure 3 illustrates this on one topology and is explicitly not evidence of a general effect (Section 7.3).
   - *Relationship-level criticality* (I_{edge}, Eq. (13)) scores individual dependencies rather than components, which is what circuit-breaker or bulkhead placement actually requires, and which a node ranking cannot express. We report this as a property of the formulation rather than an evaluated result: this paper defines the edge oracle and the model's edge head but presents no evaluation of edge-level predictions against it, so the capability is available and untested.
   - *QoS-conditioned ranking.* The 16-D edge encoding improves out-of-distribution ranking by $+0.054$ within the typed architecture (11 of 12 folds, $p = 0.0093$) and $+0.088$ within the untyped one, so declared middleware contracts carry signal a purely structural score discards.

A gate we can no longer recommend. An earlier version of this work proposed a tiered gate: run the learned model by default, and fall back to `Topo-QoS` when the model's own prediction dispersion $\hat{\sigma}$ fell below a threshold, on the evidence that $\hat{\sigma}$ tracked transfer quality. That mechanism does not survive the present corpus. Across twelve folds, $\hat{\sigma}$ correlates with the margin over `Topo-QoS` at $\rho_s = -0.126$ for HGT-QoS — the wrong sign — and the relationship is significant only for the untyped baseline (Section 7.2.1). We withdraw the recommendation rather than restate it with softer wording, and we report the withdrawal because a label-free confidence signal is exactly the kind of claim a deployment would act on.

What remains is a weaker but honest deployment position. Both engines are cheap enough to run together (Section 7.5), the ranking they produce agrees on most architectures, and the folds where they disagree are the ones a practitioner most needs warning about — with no reliable way, on present evidence, to tell which those are in advance. Running both and escalating disagreement to human review is the procedure our results support; automatic arbitration between them is not.

Role of the Explanation Layer. The RM attribution profile ($Q(v)$, Section 5) provides transparent, standards-compliant architectural diagnostics aligned with ISO/IEC 25010. By separating single-point-of-failure exposure (Availability) from wide error propagation reach (Fault Tolerance), RM provides qualitative remediation guidance (e.g., distinguishing whether a component requires replication or decoupling) that purely numeric rankers and simulation oracles cannot provide.

8.2. Performance and Computational Sustainability Implications

What the cost measurements do and do not establish. This special issue asks about the sustainability of AI techniques for software systems, and an earlier version of this work answered with a claim we can no longer support: that static graph-based gating delivers an orders-of-magnitude reduction in execution cost relative to dynamic simulation. Measurement contradicts it. On our own corpus and hardware, a five-seed `FaultInjector` sweep costs 2.5–6.3 s per scenario while the static gate costs 0.04–82.7 s; on the largest architecture the gate is roughly thirteen times the more expensive of the two (Section 7.5.1). We withdraw the efficiency claim rather than qualify it.

Three narrower statements survive measurement, and they are what we offer.

First, *within* the pipeline the learned component is negligible. The HGT forward pass is 56 ms on a 2,000-component system against 239 s for deterministic structural feature extraction, a ratio of 4,259×. Whatever the cost of pre-deployment dependability analysis turns out to be, adding a graph neural network to it is not what makes it expensive — a result that matters for anyone weighing whether learned models are affordable in a CI context.

Second, the cost is concentrated in one metric and is a deliberate accuracy purchase. Connectivity Degradation Index dominates because it is computed for every node in the main component rather than for articulation points alone; gating it would restore roughly an order of magnitude of speed at the price of a degenerate Availability score (Section 7.5). A deployment that needed the speed more than the SPOF sensitivity could make the opposite trade, and we would rather document that choice than hide it behind a faster number.

Third, and most durably, the resource the framework actually eliminates is *infrastructure*, not CPU seconds. Chaos engineering, hardware-in-the-loop benches and staging-cluster fault injection require a provisioned, running system; a manifest-time analysis requires a file. That is a real difference in what must exist before the analysis can run, and it is the honest form of the sustainability argument — but we have not measured a chaos-engineering baseline, so its magnitude remains an assertion rather than a result. Quantifying it, ideally with hardware energy counters rather than wall-clock time, is the measurement this thesis needs and does not yet have (Section 8.4).

A note on avoided runtime failure. It is tempting to argue that preventing cascading failures saves the datacenter compute that retry storms and restart loops would have consumed. We find the argument plausible and have no evidence for it: nothing in this study observes a production incident, an averted one, or the compute either would involve. We flag it as motivation rather than contribution.

8.3. Threats to Validity

Construct Validity. Our primary ground-truth impact oracle $I^*(v)$ is derived from discrete-event cascade simulation on structural models rather than observing live production outages. To evaluate construct divergence, we compared $I^*(v)$ against the queue-flow discrete-event simulator $I_{\text{dyn}}(v)$ and the multi-criteria composite oracle $I_{\text{comp}}(v)$ across diverse topologies (Table 10). The rank correlation between I^* and I_{dyn} is high ($\rho = 0.907$), but we are careful about how much it establishes: both oracles traverse the same structural graph under the same failure semantics, differing in how they score the consequences rather than in what they model, so their agreement is close to a consistency check and is weak evidence of construct validity. The more informative comparison is with I_{comp}, which differs in construct — and it agrees with I^* at only $\rho = 0.425$. That is the figure that should temper confidence here, particularly because I_{comp} supplies the labels for Table 11. Discrepancies in top-K critical-set Jaccard (0.24–0.49) stem from discrete thresholding sensitivity in non-linear cascades and tied zero-inflation, which we analyse explicitly in Supplementary Section S2. Convergent validity across simulation paradigms is therefore partially supported at the level of ordering and not established at the level of critical-set membership; no oracle in this study is validated against an observed production failure (Section 4.4). For Maintainability, $I_M(v)$ traverses the same derived dependency topology from which $M(v)$ is scored; independent validation against repository change histories or version-control churn remains future work.

Internal Validity. Potential feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis}, whereas ground-truth simulation oracles operate on $G_{\text{structural}}$, formally asserted in continuous integration. Substrate parity is rigorously maintained: learned models (HGT-QoS, GAT-N-QoS) share identical training sets, depths, and held-out scenario early-stopping protocols. In the QoS schema, four dimensions (reliability, durability, transport priority, heterogeneity flag) capture active operational middleware configurations, while three dimensions (deadline, max blocking) are reserved extension points that remain zero in standard static descriptors.

External Validity. Our evaluation spans twelve synthetic architectures and five authentic open-source distributed systems, and the boundary it establishes is sharper than the coverage suggests. Learned transfer to the real systems is *not* demonstrated: restricted to components that actually propagate failures, mean rank correlation falls to +0.180 and inverts on the two microservice call-tree architectures (Section 7.4.1). Every synthetic scenario in the training corpus is a pub-sub mesh emitted by one parameterized generator, so a model trained on it has never seen a deep synchronous RPC hierarchy, and the two systems it fails on are exactly those. Corpus diversity, not corpus size, is the limiting factor, and no amount of additional generated pub-sub topologies would address it.

A second limit concerns QoS coverage. An earlier version of the generator emitted near-constant QoS profiles, which would have made the QoS ablation of Section 7.3.1 uninformative; the corpus reported here carries genuine variation (modal shares 29–89%), but all of it is generator-produced, and we have no evidence about the distribution of QoS declarations in real deployments. Finally, the largest system we evaluate has 520 components and the largest we time has 2,000; the architectures motivating this work in Section 1.1 are an order of magnitude larger, and the dominant pipeline stage is $O(|V|^2 + |V||E|)$, so neither the accuracy nor the cost results should be extrapolated to that scale.

Conclusion Validity. Given heavy-tailed impact distributions, statistical analyses use non-parametric rank correlation (Spearman ρ, Kendall τ), bootstrap confidence intervals ($B = 2{,}000$) and paired Wilcoxon signed-rank tests, with the fold or scenario as the unit of analysis. Two hazards recur and shape how we report. Pooling across heterogeneous entity types triggers Simpson's paradox — pooled $\rho = 0.057$ sits below every per-type value it aggregates (0.149–0.515) — so all headline figures are stratified on a single population. And rank correlation over zero-inflated labels conflates ordering the active components with separating them from inert ones, which is why we report zero-excluded correlations alongside full-population ones wherever the label distribution permits. Where the two disagree, as on the real-world systems, we read the zero-excluded figure as the ranking result.

8.4. Limitations and Future Work

An incomplete baseline on the real-world systems. `Topo-QoS` — the strongest training-free baseline on the synthetic corpus, and the one whose competitiveness bounds our central claim — is not scored in Table 12. It requires QoS-weighted betweenness recomputed on the projection graph, which the real-world evaluation cache does not carry, and we chose to leave the cell empty rather than publish an unreproducible figure beside reproducible ones. The consequence is that the real-world comparison is made against the weaker `Topo`, and the margin reported there should be read accordingly. Closing this gap requires only extending the cache, and it is the first thing we would add.

The explanation layer is not validated as an explanation. SaG's attribution layer separates Availability from Fault Tolerance on the claim that they imply different repairs. Nothing in this paper tests that claim. We do not show that the two sub-scores rank components differently in practice, that practitioners find the distinction actionable, or that applying a recommended repair reduces measured failure impact. The elicited AHP weights, meanwhile, are measurably worse than a uniform prior at ranking (Section 7.3), and while we argue that ranking is not what they are for, we have no positive evidence for what they *are* for. Three studies would settle this: a discriminant analysis of $A(v)$ against $FT(v)$ over the corpus, a counterfactual evaluation in which recommended edits are applied and re-simulated, and a practitioner study of whether the diagnoses change what an engineer would do. Until then, the layer's contribution is a principled mapping, not a validated instrument.

No label-free reliability signal. We previously reported that the model's prediction dispersion flagged the architectures it ranked poorly, which would have made the predictor safe to deploy behind an automatic fallback. That relationship does not hold on the present corpus (Section 7.2.1). A practitioner applying the model to an unseen architecture currently has no reliable indication of whether it is one of the cases where a closed-form baseline would serve better. Finding such a signal is, in our view, more valuable to the pre-deployment setting than any further gain in mean correlation.

Distributed AI and LLM Serving Topologies. Modern AI infrastructure relies on distributed LLM serving systems (e.g., vLLM, Triton, DeepSpeed) characterized by complex tensor, pipeline, and expert parallelism across GPU clusters, dynamic KV-cache routing, and disaggregated prefill-decode architectures. A straggler or failing GPU in a pipeline-parallel ring induces severe head-of-line blocking and massive GPU idle power dissipation. Extending the SaG multigraph schema to model distributed AI serving topologies (representing GPU worker nodes, tensor communication channels, and KV-cache transfer fabrics) offers a high-impact direction for sustainable AI systems engineering.

Empirical Power and Hardware Testbeds. Validating the sustainability thesis through empirical power measurements using running package counters (Intel/AMD RAPL, NVIDIA NVML) and IPMI sensors on Kubernetes clusters during live fault injection, alongside Hardware-in-the-Loop (HIL) validation on cyber-physical testbeds (e.g., ROS 2 CAN-bus autonomous driving compute platforms).

Automated Refactoring and Self-Healing. Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies, insert circuit-breakers, and add redundant broker pathways to eliminate single points of failure.

8.5. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that combines a relation-specific Heterogeneous Graph Transformer for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability attribution layer, both operating on a typed multigraph derived from Architecture-as-Code manifests with no runtime telemetry.

The central empirical finding is about *when* architectural typing pays. Given training data from the same distribution as the test split, typed and untyped message passing are indistinguishable, and the best in-distribution mean belongs to the untyped model. Asked to rank an architecture it has never seen, the typed model wins 11 of 12 inductive folds by $\Delta\rho = +0.114$ ($p = 0.0122$), and the 16-D QoS edge encoding adds a further $+0.054$ ($p = 0.0093$) independently of typing. Relation-specific parameters therefore act as an inductive bias for unfamiliar topologies rather than as additional fitting capacity — which is the regime a pre-deployment gate always operates in, since every architecture it sees is by construction new.

We are equally clear about what is not established. Against an unparameterized QoS-weighted centrality score, learned ranking is not superior: the nominal margin of $+0.127$ ($p = 0.077$) rests almost entirely on a single fold and falls to $+0.078$ without it. On the five open-source systems, full-population correlation of $\rho = 0.682$ drops to $+0.180$ once restricted to components that actually propagate failures, and inverts on both microservice call-tree architectures, so zero-shot transfer to authentic systems is a negative result on this evidence. A label-free confidence signal we previously reported does not replicate. The explanation layer's elicited AHP weights measurably worsen ranking relative to a uniform prior, and we have no independent evidence that they improve attribution — validating the attribution itself remains its principal open question.

What the work does contribute is a reproducible typed-multigraph formulation of pub-sub architecture, a corpus that regenerates byte-identically from committed configurations, a clean measurement of where relational typing helps and where it does not, and a pre-deployment analysis pipeline whose learned component is the cheapest stage by three orders of magnitude. Whether that pipeline predicts failures that actually occur — rather than failures a simulator produces — is the question we most want answered next, and it requires field data no static corpus can supply.

Declarations

CRediT Authorship Contribution Statement. Ibrahim Onuralp Yigit: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision, Project administration.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Supplementary Material. The parameter-sensitivity analyses supporting RQ3 — one-factor and Morris global screening over the explanation layer's ten declared weight constants, the cascade-threshold and normalisation sweeps, and the zero-inflation sensitivity of the inter-oracle agreement figures — are provided as a supplementary document (Supplementary Sections S1–S2). Their conclusions are summarised in Section 7.3; the supplement reports the full sweeps.

Data Availability. The complete replication package—including synthetic scenario datasets, generator configurations, simulation harnesses, real-world architecture adapters, trained model checkpoints, and all analysis scripts—is openly available on Zenodo under DOI 10.5281/zenodo.14922108 and cited as [85] in compliance with Option C of the Elsevier research data policy. The synthetic corpus is regenerable: each dataset carries its random seed and SHA-256 cryptographic digest in a committed manifest, with automated tests asserting byte-identical regeneration from configuration files (Section 6.1.1). Every table and figure is produced deterministically from committed artifacts by reproducible scripts; none of the reported values is transcribed manually.

Declaration of Generative AI and AI-assisted Technologies in the Manuscript Preparation Process. During the preparation of this work, the authors used AI-assisted language tools to check grammar, improve readability, and support LaTeX typesetting. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, Science Robotics 7 (66) (2022) eabm6074.

[2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: Proc. 6th Int. Workshop on Networking Meets Databases (NetDB), 2011.

[3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).

[4] OASIS, MQTT version 5.0, OASIS Standard (2019).

[5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: Present and Ulterior Software Engineering, Springer, 2017, pp. 195–216.

[6] S. Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2015.

[7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, ACM Computing Surveys 35 (2) (2003) 114–131.

[8] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, IEEE Transactions on Dependable and Secure Computing 1 (1) (2004) 11–33.

[9] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).

[10] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

[11] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.

[12] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.

[13] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.

[14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.

[15] SonarSource, Clean as you code, SonarQube documentation (2024).
[16] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.
[17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.
[18] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.
[19] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.
[20] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.
[21] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
[22] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.
[23] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.
[24] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE), 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
[25] R. C. Cheung, A user-oriented software reliability model, IEEE Transactions on Software Engineering SE-6 (2) (1980) 118–125.
[26] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, Performance Evaluation 45 (2–3) (2001) 179–204.
[27] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, Software and Systems Modeling 7 (1) (2008) 49–65.
[28] S. Becker, H. Koziolek, R. Reussner, The Palladio component model for model-driven performance prediction, Journal of Systems and Software 82 (1) (2009) 3–22.
[29] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, IEEE Transactions on Software Engineering 35 (2) (2009) 148–161.
[30] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2019.
[31] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.
[32] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2020.
[33] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS), 2021.
[34] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2022.
[35] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE), 2023.
[36] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.
[37] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
[38] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.

[39] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.
[40] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
[41] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
[42] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
[43] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
[44] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
[45] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
[46] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).
[47] T. L. Saaty, The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation, McGraw-Hill, 1980.
[48] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, Nature 406 (2000) 378–382.
[49] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, Physical Review E 66 (2002) 065102(R).
[50] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, Nature 464 (2010) 1025–1028.
[51] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, Nature Machine Intelligence 2 (2020) 317–324.
[52] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM), 2019, pp. 559–568.
[53] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
[54] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2017.
[55] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 1024–1034.
[56] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
[57] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: Proc. European Semantic Web Conference (ESWC), 2018, pp. 593–607.
[58] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: Proc. The Web Conference (WWW), 2019, pp. 2022–2032.
[59] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
[60] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: Proc. The Web Conference (WWW), 2020, pp. 2331–2341.
[61] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 9244–9255.

[62] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 19620–19631.
[63] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference, Morgan Kaufmann, 1988.
[64] G. Beliakov, A. Pradera, T. Calvo, Aggregation functions: A guide for practitioners, Studies in Fuzziness and Soft Computing 221 (2007).
[65] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, IEEE Transactions on Systems, Man, and Cybernetics 18 (1) (1988) 183–190.
[66] G. H. Hardy, J. E. Littlewood, G. Pólya, Inequalities, 2nd Edition, Cambridge University Press, 1952.
[67] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
[68] R. C. Martin, Agile Software Development: Principles, Patterns, and Practices, Prentice Hall, 2003.
[69] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
[70] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
[71] Team SimPy, Simpy: Event discrete simulation for Python, `https://simpy.readthedocs.io` (2020).
[72] International Organization for Standardization, ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use, Tech. rep., International Organization for Standardization (2016).
[73] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS), 2018, pp. 287–296.
[74] Google Cloud Platform, Online boutique: A cloud-native microservices demo application, Software artifact (2024).
[75] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, IEEE Transactions on Software Engineering 47 (2) (2021) 243–260.
[76] Home Assistant Community, Home assistant: Open source home automation that puts local control and privacy first, Software artifact, `https://www.home-assistant.io/` (2024).
[77] Linux Foundation LF Edge, Edgex foundry: An open, vendor-neutral edge iot middleware platform, Software artifact, `https://www.edgexfoundry.org/` (2024).
[78] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.
[79] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.
[80] C. Spearman, The proof and measurement of association between two things, American Journal of Psychology 15 (1) (1904) 72–101.
[81] M. D. Morris, Factorial sampling plans for preliminary computational experiments, Technometrics 33 (2) (1991) 161–174.
[82] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, Environmental Modelling & Software 22 (10) (2007) 1509–1518.
[83] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, Global Sensitivity Analysis: The Primer, John Wiley & Sons, 2008.
[84] I. M. Sobol', Sensitivity estimates for nonlinear mathematical models, Mathematical Modelling and Computational Experiments 1 (4) (1993) 407–414.
[85] I. O. Yigit, F. Buzluca, Software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), `https://doi.org/10.5281/zenodo.14922108` (2026). `doi:10.5281/zenodo.14922108`.